

DESIGN PATTERN

A COSA SERVONO

I design pattern come già sappiamo sono delle strutture di codice che forniscono soluzioni strutturali collaudate per progettazioni software, e quindi costruiscono una struttura software solida e chiara che permette di raggruppare più classi di un programma.

Ovviamente se i requisiti del programma software dovessero cambiare nel tempo, il design pattern struttura le classi in modo che siano modificabili, estendibili, e riutilizzabili nel tempo.

Nominare un pattern ovviamente permette ad un intero team di comprendere un architettura senza doverla spiegare.

COME SI APPLICANO

I pattern non sono linee di codice pronte da dover installare nel programma, ma sono linee di guida astratte che permettono di comprendere al meglio il codice. Si utilizzano quando si vuole impostare al meglio il programma, oppure in fase di refactoring, così da creare interfacce e classi astratte che permettono di dare la giusta ereditarietà a tutte le componenti del programma.

Ogni pattern possiede una sua carta di identità, contenente:

- Nome**: permette di dare un'idea indicativa sullo scopo del pattern
- Intento**: descrivere lo scopo del design pattern
- Problema**: descrive il problema a cui è applicato il design pattern e le condizioni necessarie per applicarlo
- Soluzione**: descrive le classi, le loro responsabilità e le loro relazioni
- Conseguenze**: indicano i risultati, vantaggi e svantaggi nell'uso del design pattern

CATEGORIE DESIGN PATTERN

Di design pattern ce ne sono di diverse categorie, e sono:

PATTERN CREAZIONALI->

I pattern creazionali sono coloro che svolgono il lavoro della creazione di nuovi oggetti all'interno del programma, ma ovviamente in modo semplice e comodo. Normalmente gli oggetti in un programma vengono creati tramite la parola magica new, ad esempio: new Automobile(). Ma di questo passo creando tanti oggetti con la parola new, si andrebbe a riempire tutto il programma, e se solo si volesse modificare un'istanza di tale oggetto si dovrebbero andare a modificare 100 mila punti diversi. Con i pattern creazionali invece questo non succede.

VANTAGGI:

- Chi vuole creare l'oggetto non deve più stare a chiedersi come crearlo, ma richiede semplicemente l'oggetto già creato;
- Il tuo programma non dipende più dalle classi concrete dove si crea l'oggetto, ma dipende dall'interfaccia, e poi sarà il pattern a decidere quale classe istanziare;
- Può imporre un numero limite di oggetti da creare;

TIPI DI PATTERN->

Di design pattern creazionali ne abbiamo di 4 tipi:

- SINGLETON;
- FACTORY METHOD;
- PROTOTYPE;
- OBJECT POOL;

SINGLETON->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Garantisce che esista una e una sola copia di un oggetto in tutto il programma.

SCOPO:

Il singleton è il pattern dell' "uno e solo uno, con costruttore privato", ossia che si assicura che di una determinata classe, esista una e una sola copia di un'istanza in tutto il programma, fornendo a tutti un'unico punto d'accesso per usarla.

COME RICONOSCERLO:

Il singleton è riconoscibile facilmente dal costruttore, perché dato che deve essere creata un'unica istanza di quella classe, il costruttore deve essere impostato privato, quindi se qualcuno prova a creare una nuova istanza con lo stesso nome, il compilatore blocca tutto.



Come vediamo nel diagramma UML sopra citato vediamo come abbiamo l'unica istanza tenuta nella variabile(uniqueInstance) che anch'essa è contrassegnata dal meno, ciò significa che è unica.

Poi avremo il metodo getInstance() contrassegnato con il +, perché sarà l'unico punto di accesso pubblico dell'istanza.

CODICE ESEMPIO:

```
public class RegistroDiSistema {

    // 1. LA VARIABILE NASCOSTA: Contiene l'unica copia esistente
    private static RegistroDiSistema istanzaUnica;

    // Una variabile di prova per dimostrare che l'oggetto è unico
    private String volume;

    // 2. IL COSTRUTTORE PRIVATO: Il "lucchetto"
    // Mettendo "private", impediamo a chiunque di fare "new RegistroDiSistema()"
    private RegistroDiSistema() {
        this.volume = "Medio"; // Impostazione di base
        System.out.println("\ Registro creato fisicamente per la prima e unica volta!")
    }

    // 3. IL PORTONE D'INGRESSO: L'unico modo per avere l'oggetto
    public static RegistroDiSistema getInstance() {
        // Se è la primissima volta che qualcuno lo chiede, lo creiamo
        if (istanzaUnica == null) {
            istanzaUnica = new RegistroDiSistema();
        }
        // Lo restituiamo (sia che l'abbiamo appena creato, sia che esistesse già)
        return istanzaUnica;
    }

    // --- Metodi normali della classe ---
    public String getVolume() { return volume; }
    public void setVolume(String nuovoVolume) { this.volume = nuovoVolume; }
}
```

```
public class Main {
    public static void main(String[] args) {

        // ✗ ERRORE DI COMPILAZIONE:
        // RegistroDiSistema errato = new RegistroDiSistema();
        // (Il compilatore ti blocca perché il costruttore è privato!)

        System.out.println("--- FASE 1: Chiedo il registro per la prima volta ---");
        RegistroDiSistema registroA = RegistroDiSistema.getInstance();
        System.out.println("Volume di A: " + registroA.getVolume());

        System.out.println("\n--- FASE 2: Modifico il volume da A ---");
        registroA.setVolume("ALTO");
        System.out.println("Volume di A impostato su ALTO.");

        System.out.println("\n--- FASE 3: Chiedo il registro una SECONDA volta ---");
        // Noterai che la scritta "\ Registro creato fisicamente..." NON comparirà di nuovo
        RegistroDiSistema registroB = RegistroDiSistema.getInstance();

        // LA PROVA DEL NOVECENTO:
        // Se chiedo il volume a B, mi risponderà "ALTO", perché B e A
        // sono lo stesso identico spazio in memoria!
        System.out.println("Volume letto da B: " + registroB.getVolume());
    }
}
```

Qui vediamo il codice esempio, di come viene creata un'istanza unica che è accessibile da tutti, ma se nel main provassi a creare un'altra istanza mi darebbe errore di compilazione, semplicemente perché il COSTRUTTORE È PRIVATO. Quindi in poche parole succede che, non crea una nuova istanza con un nuovo costruttore con parametri diversi, ma crea un'altra variabile che punta sempre a quella stessa istanza con lo stesso costruttore.

FACTORY METHOD->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO

Nasconde il `new` al programma principale. Delega a una "Fabbrica" la decisione su quale classe specifica istanziare (es. la fabbrica di fattorini che decide se mandare bici o camion).

- **Cosa fa:** Definisce un'interfaccia o un metodo per creare un oggetto, ma delega alle sottoclassi (o alla logica interna del metodo stesso) la decisione esatta su *quale* classe specifica istanziare.
- **Come agisce:** Centralizza la creazione degli oggetti nascondendo al client l'uso diretto dell'operatore `new`. Il client chiede un oggetto chiamando la "fabbrica" e riceve un'istanza pronta all'uso. Il client conosce solo l'interfaccia generale dell'oggetto restituito, ignorando i dettagli di come è stato costruito.

SCOPO:

Il factory method è il pattern più utilizzato e famoso, e serve per creare oggetti senza dover specificare l'esatta classe dell'oggetto che verrà creato. Quindi in poche parole invece che dare il potere a tutte le classi del programma di creare nuovi oggetti con la parolina magica `new`, che verrà sparsa per tutto il programma, diamo il privilegio solamente alla classe fabbrica. Quindi quando il programma principale richiede un'oggetto, a lui non interessa da quale classe prenderlo, ma ha solamente un metodo per richiedere l'oggetto, e tale oggetto sarà istanziato dalla classe fabbrica.

COME RICONOSCERLO:

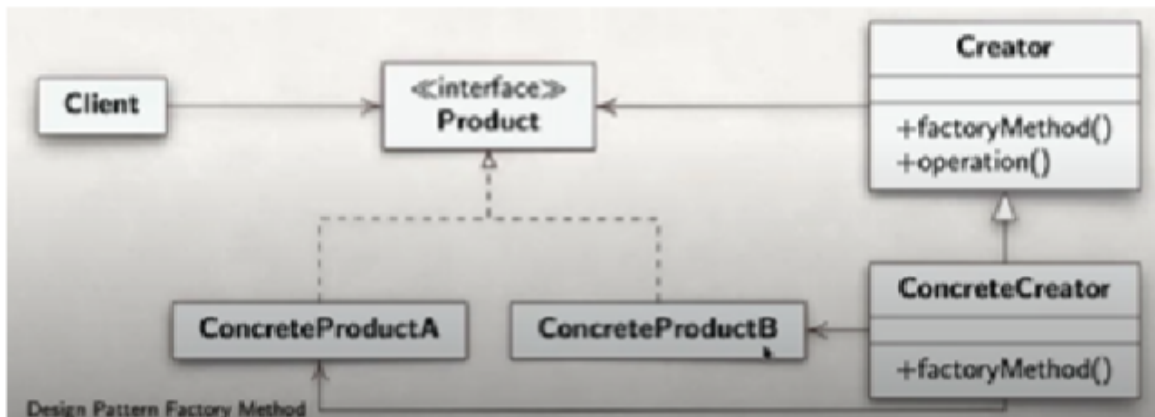
Partendo dall'esempio di un'azienda che effettua consegne, e possiede auto moto e bici, abbiamo 4 componenti fondamentali del factory method:

-PRODUCT-> È l'interfaccia o la classe astratta generale, quindi è l'unica cosa che il cliente conosce, e in questo caso sarebbe `MezzoDiTrasporto()`. Sa che qualunque cosa uscirà dalla fabbrica, avrà un metodo `consegna()`.

-CONCRETE PRODUCT-> È il prodotto vero e proprio che esce dalla fabbrica(PRODUCT), ed è auto,moto,bici. Sono le classi specifiche che implementano il prodotto `base(PRODUCT)`.

-CREATOR-> È la classe astratta che contiene il cuore del pattern `FactoryMethod`, ossia che contiene l'idea di creare il mezzo, ma non sa quale mezzo verrà creato;

-CONCRETE CREATOR-> Il concrete creator sono le sottoclassi del creator, che implementano la creazione dei mezzi, facendo `override` del `factory method`.



VANTAGGI:

Il vantaggio vero è che si è separato chi usa l'oggetto da chi lo crea, così che se si dovranno aggiungere altri oggetti in futuro basterà aggiungere un concrete creator in più.

Ci sono delle varianti del factory, che aiutano a renderlo più snello, come:

- Classe unica, invece che fare creator e concrete creator, si crea un'unica classe che fa tutto;

- Rendere il metodo `factoryMethod()` statico, ossia che il cliente non deve fare `new Fabbrica()`, ma semplicemente `Fabbrica.creaMezzo()`, che è molto più comodo.

DEPENDENCY INJECTION-> Questa iniezione di dipendenza, ci dice che se un'oggetto ha bisogno di altri oggetti esterni per funzionare, allora si iniettano direttamente nel costruttore di tale oggetto, in modo tale che li ha sempre con se, e se si pensa di fare `new oggetto()`, è sbagliato perché altrimenti quell'oggetto sarebbe attaccato a quello specifico oggetto, e non ad uno che scegliamo noi sul momento della creazione dell'oggetto principale.

```
// Il Camion non fa "new", riceve il motore già costruito da fuori
public Camion(Motore motoreGiaPronto, Autista autistaGiaPronto) {
    this.motore = motoreGiaPronto;
    this.autista = autistaGiaPronto;
}
```

CODICE:

```
// =====
// 1. PRODUCT
// =====
// Questa è l'interfaccia comune. Il Client (il Main) userà solo questa,
// senza sapere quale mezzo reale si nasconde dietro.
interface Mezzo {
    void consegna();
}

// =====
// 2. CONCRETE PRODUCT (A)
// =====
// Primo prodotto reale che implementa l'interfaccia Product.
class Bicicletta implements Mezzo {
    @Override
    public void consegna() {
        System.out.println("🚲 Consegna in corso tra i vicoli del centro con la Bicicletta");
    }
}

// =====
// 2. CONCRETE PRODUCT (B)
// =====
// Secondo prodotto reale che implementa l'interfaccia Product.
class Camion implements Mezzo {
    @Override
    public void consegna() {
        System.out.println("🚚 Consegna in corso sulla superstrada con il Camion!");
    }
}

// =====
// 3. CREATOR
// =====
// La classe di base che gestisce la logica del servizio.
// Dichiarare il magico "Factory Method" ma NON sa quale mezzo verrà creato.
abstract class PianificatoreTrasporti {

    // ECCO IL FACTORY METHOD astratto.
    // Dice solo: "Chi mi estende deve sputare fuori un oggetto di tipo Mezzo".
    public abstract Mezzo factoryMethod();

    // Metodo di lavoro normale che usa il mezzo generico
    public void avviaSpedizione() {
        // Chiamo il metodo fabbrica interno per farmi dare un mezzo
        Mezzo ilMioMezzo = factoryMethod();

        // Uso il mezzo fregandomene di cosa sia di preciso (Polimorfismo!)
        ilMioMezzo.consegna();
    }
}

// =====
// 4. CONCRETE CREATOR (A)
// =====
// Fabbrica specializzata che decide di fabbricare Biciclette.
class PianificatoreUrbano extends PianificatoreTrasporti {
    @Override
    public Mezzo factoryMethod() {
        return new Bicicletta(); // Istanza il Concrete Product A
    }
}
```

PS:

Nel codice che ho fatto io su VS code notiamo come ho creato una classe astratta iniziale dove inserisco il costruttore, delle variabili comuni a tutte le classi di oggetti che andrò a creare, e poi funzioni in comune tra tutte le classi e funzioni che ogni classe implementerà per se. Dopo aver implementato tutto con le classi, che per pulizia dovranno essere implementate ognuna nel proprio file, nel main andrò a creare un nuovo oggetto del tipo di quella classe, quindi se la classe si chiama mario, allora sarà: `mario m= new mario()`, e quindi non andremo a creare un nuovo oggetto, ma una nuova istanza della classe, con la quale potremo andare a chiamare la funzione generale nella classe astratta iniziale, che richiamerà tutte le funzioni implementate da quella specifica classe.

OBJECT POOL->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Un "magazzino" per riciclare oggetti pesanti da caricare. Invece di crearli e distruggerli, li prendi in prestito e li restituisci (es. le connessioni al database).

SCOPO:

L'object pool o piscina di oggetti, è una depository di istanze, che contiene tante istanze che vengono fornite ai client, e che questi ultimi utilizzano più volte, e quando hanno finito di utilizzare una certa istanza, la rimetteranno nella depository. Quindi invece che creare un'oggetto con `new` e distruggerlo subito dopo l'utilizzo, tale oggetto verrà conservato in questa depository. Un'object pool potrebbe essere un'array, una lista ecc.. . Questa strategia viene usata con oggetti più grandi, perché giustamente allocare memoria, inizializzare variabili costa in termini di processore, e quindi si fa una volta e poi questi oggetti vengono conservati.

COME RICONOSCERLO:

Si riconosce facilmente, perché è una struttura dati che contiene tantissime istanze che possono essere riutilizzate da tutti i client nel programma.

VANTAGGI

I vantaggi sono soprattutto in termini di memoria e velocità del processore, ma anche che ogni client ha un uso esclusivo dell'istanza, perché quando un client usa un'istanza contenuta nella depository, la può usare solo lei in quel preciso momento. Si adatta perfettamente con il `factory method`. Questa depository può essere a dimensione fissa o variabile.

CODICE:

```
public class CreatorPool extends ShapeCreator{
    private LinkedList<Shape> pool = new LinkedList<>();

    public Shape getShape(){
        Shape s;
        if(pool.size()>0) s = pool.remove();
        else s = new Circle();
        return s;
    }

    public void releaseShape(Shape s){
        pool.add(s);
    }
}
```

Guardiamo l'esempio in Java scritto nella slide: la classe si chiama `CreatorPool` (che estende una Factory) e gestisce un pool di forme geometriche (`Shape`). Ha due metodi speculari che fanno tutto il lavoro:

- **Il metodo per prendere l'oggetto (`getShape()`):** Quando chiedi una forma, il pool controlla la sua lista (`pool.size() > 0`):
 - **Se c'è un oggetto disponibile nel deposito:** lo estrae (`pool.remove()`) e te lo passa. Non è stato usato nessun `new`, l'oggetto è riciclato!
 - **Se il deposito è vuoto:** allora (e solo in quel caso) è costretto a fare un `new Circle()` per crearne uno nuovo di zecca.
- **Il metodo per restituire l'oggetto (`releaseShape(Shape s)`):** Quando hai finito il tuo lavoro con la forma geometrica, chiami questo metodo passandogli l'oggetto. Il pool prende la forma e la reinserisce nella lista (`pool.add(s)`), pronta per il prossimo cliente.

PROTOTYPE->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Invece di creare un oggetto da zero impostando mille parametri, fai la "fotocopia" veloce (clona) di un oggetto già esistente.

SCOPO:

Il pattern prototype è un pattern che invece di far creare un'oggetto da zero, quindi passando per costruttori complessi e reimpostare tutti i parametri, prende un'oggetto già esistente, e ne fa un'esatta fotocopia. Questo oggetto di partenza sarà il prototipo per tutti gli altri.

COME RICONOSCERLO:

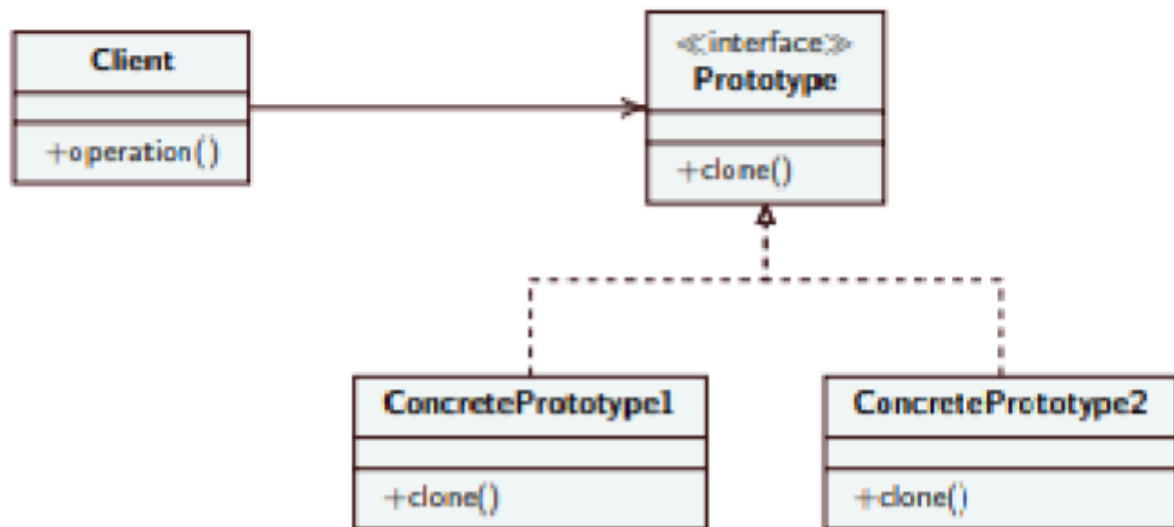
Il prototype è facile da riconoscere nel codice, perché tutto il codice ruota attorno ad un unico metodo, ossia `clone()`. C'è sempre un'interfaccia base(prototype) che obbliga le classi concrete a possedere questo metodo, il cui unico scopo è restituire copie di se stesse. È

DIVISO IN:

-PROTOTYPE-> che è un'interfaccia pubblica che contiene il metodo per clonare le classi che ereditano tale interfaccia;

-CONCRETE PROTOTYPE-> È il cuore del pattern, perché è la classe che implementa l'interfaccia, e crea una copia di se stessa clonando con il costruttore;

-CLIENT-> Il client è il main, che crea l'oggetto, e ne crea la copia tramite il metodo contenuto nel concrete prototype.



VANTAGGI:

- **Disaccoppiamento:** Il Client usa il metodo `clone()` sull'interfaccia generica. Non ha bisogno di conoscere la classe concreta dell'oggetto che sta copiando.
- **Creazione a Runtime:** Puoi aggiungere nuovi prototipi da clonare mentre il programma sta già girando.
- **Efficienza:** Se un oggetto richiede calcoli pesanti o lunghe letture dal database per essere creato la prima volta, clonarne uno già pronto in memoria è un'operazione quasi istantanea.

CODICE:

```
// 1. L'Interfaccia (Prototype)
public interface Prototipo {
    Prototipo clona();
}

// 2. L'Oggetto Concreto (ConcretePrototype)
public class Modulo implements Prototipo {
    private int id;
    private String nome;

    public Modulo(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    // IL CUORE DEL PATTERN: L'oggetto sa come copiare se stesso
    @Override
    public Prototipo clona() {
        // Crea una nuova istanza passando i propri dati attuali
        return new Modulo(this.id, this.nome);
    }
}

// 3. Il Client (Main)
public class Client {
    public static void main(String[] args) {
        // 1. Creo il primo oggetto a mano (il Prototipo)
        Modulo moduloOriginale = new Modulo(8, "CompactModule");

        // 2. Me ne serve un altro identico. Non compilo i dati da zero, lo clono!
        Prototipo moduloCopiato = moduloOriginale.clona();
    }
}
```

Come vediamo dal codice, come abbiamo spiegato prima abbiamo l'interfaccia(prototipo) che contiene il metodo per clonare. Poi la classe che implementa l'interfaccia(concrete prototype) che implementa il metodo clona tramite @Override. E poi il main(client) che implementa il metodo clona e crea l'oggetto.

PATTERN STRUTTURALI->

Dopo aver capito come creare oggetti in modo semplice ed efficace, ora capiamo come assemblarli. Si occupano della composizione di classi e oggetti, e si occupano di unire pezzi diversi di codice tra loro, per formare un codice efficiente e pulito. Quindi funzionano come adattatori che riescono a far lavorare insieme strutture diverse. Immaginiamo i pattern strutturali come l'officina meccanica del nostro software. Immagina di avere un pezzo di un kart nuovo, ma i suoi attacchi non combaciano con il telaio. Invece che saldare e tagliare a caso, costruisci una staffa intermedia che permette di farli adattare, questi sono i pattern strutturali. I pattern strutturali:

- Risolvono l'incompatibilità: Fanno comunicare tra loro oggetti e classi che normalmente non potrebbero mai comunicare, e lo fanno tramite l'adapter.

- Semplificare la Complessità:** Prendono un sottosistema enorme e incasinato (pensa a tutte le valvole, candele e centraline di un'auto) e lo nascondono dietro a un'unica interfaccia semplicissima (un bottone "Start"), così che il resto del programma non debba impazzire a capire come funziona il motore (questo è il Facade).

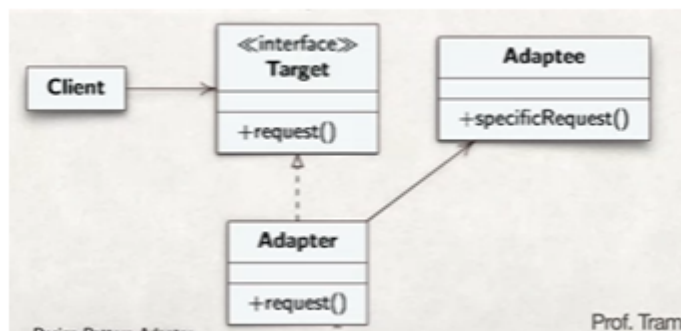
Aggiungere Flessibilità senza Ereditarietà: Ti permettono di "agganciare" nuove funzionalità a un oggetto al volo, come se stessi montando optional su un'auto, evitandoti di dover creare decine di sottoclassi infinite e rigide (questo è il compito del `Decorator` o del `Composite`).

ADAPTER->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Il "traduttore". Permette a due classi con interfacce incompatibili di comunicare, mascherando quella strana (es. la spina americana e la presa europea).

SCOPO:



L'adapter è il primo pattern di cui parlavamo, ossia quello che permette di far comunicare 2 oggetti che non parlano la stessa lingua, tra loro, e ovviamente viene impossibile modificare il codice di una o dell'altra. Quindi si crea un adattatore tra i 2 che permette di sviare i problemi di linguaggio.

COME RICONOSCERLO:

L'adapter ha 4 protagonisti, che sono:

-Target(presa Europea): Il target è l'interfaccia base che il nostro programma si aspetta di usare e che sa usare. Possiede un metodo standard che nel diagramma UML sopra si chiama `request()`.

-Client(Elettrodomestico): È il programma principale che è ignorante, e conosce solo il target. Significa che vuole solo chiamare `request()` e sperare che le cose funzionino;

-Adaptee(Spina americana): È il componente straniero che dobbiamo assolutamente usare, ma parla un'altra lingua rispetto al target, e il suo metodo si chiama `specificRequest()`;

-Adapter(Adattatore): È colui che adatta target e adaptee, ed è l'unica classe che puoi scrivere da zero.

Quindi in breve come descrive il diagramma UML, si traveste da presa europea e implementa l'interfaccia target. Nasconde la spina americana, guardando la freccia che punta verso destra, quindi significa che dentro l'adapter c'è l'adaptee. Quando il Client preme il bottone chiamando il metodo normale `request()`, l'Adapter riceve il comando, si gira di nascosto verso l'Adaptee e chiama il suo metodo strano `specificRequest()`, per poi restituire il risultato al Client.

CODICE:

1. Il Target (L'interfaccia del tuo E-commerce)

Questa è la regola che il tuo e-commerce usa da anni. Tutto il tuo carrello acquisti è programmato per usare questa precisa funzione.

Java



```
public interface SistemaDiPagamento {  
    // Il nostro sistema accetta semplicemente l'importo in Euro (es. 25.50)  
    void processaPagamento(double importoInEuro);  
}
```

2. L'Adaptee (La libreria di Stripe - Incompatibile)

Questa è la classe fornita da Stripe. Non puoi toccarne il codice sorgente. Come vedi, il metodo ha un nome diverso e richiede parametri diversi.

Java



```
public class LibreriaStripe {  
    // Stripe richiede l'importo in centesimi interi (es. 2550) e la stringa della valuta  
    public void eseguiTransazione(int importoCentesimi, String valuta) {  
        System.out.println("== [STRIPE API] Connessione al server...");  
        System.out.println("== [STRIPE API] Pagamento autorizzato di " + importoCentesimi);  
    }  
}
```

3. L'Adapter (Il tuo Traduttore)

Creiamo la classe "ponte". Implementa la TUA interfaccia, ma di nascosto usa la LORO libreria, facendo la conversione dei dati (i famosi "Euro in centesimi").

Java



```
public class StripeAdapter implements SistemaDiPagamento {

    private LibreriaStripe apiEsterna;

    // Quando creo l'adattatore, inizializzo la libreria di Stripe
    public StripeAdapter() {
        this.apiEsterna = new LibreriaStripe();
    }

    @Override
    public void processaPagamento(double importoInEuro) {
        // 1. L'Adapter fa il lavoro sporco di traduzione dei dati
        int importoCentesimi = (int) (importoInEuro * 100);
        String valutaDiDefault = "EUR";

        // 2. Chiama il metodo strano della libreria esterna
        apiEsterna.eseguiTransazione(importoCentesimi, valutaDiDefault);
    }
}
```

4. Il Client (Il tuo Main / Carrello)

Guarda quanto è pulito il codice del carrello. Al carrello non frega assolutamente nulla di come funziona Stripe o di dover convertire i soldi in centesimi. Lui chiama il "suo" metodo, e l'Adapter fa il resto.

Java



```
public class CarrelloEcommerce {
    public static void main(String[] args) {

        System.out.println("🛒 Utente al checkout. Totale carrello: 45.50€");

        // Vogliamo usare Stripe? Nessun problema, istanziamo l'Adapter.
        // Nota: lo salviamo dentro la nostra interfaccia standard!
        SistemaDiPagamento gateway = new StripeAdapter();

        // Il carrello processa il pagamento normalmente
        gateway.processaPagamento(45.50);

        System.out.println("✅ Ordine completato con successo!");
    }
}
```

FACADE->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Il "pulsante magico" (o il cameriere). Nasconde un intero sottosistema enorme e complesso dietro a un'unica interfaccia semplicissima (es. il tasto "Compra ora" su Amazon).

SCOPO:

Il facade funziona come un pulsante magico. È un pattern che nasconde tutte le complessità che stanno dietro un programma, ed evita di andare a chiedere e a cercare informazioni in tutte le classi, così creandosi molte difficoltà. Un esempio plausibile sarebbe quello del ristorante. c'è il Cuoco, l'Aiuto-cuoco, il Sommelier, il Lavapiatti). Se tu (il *Client*) dovessi andare in cucina a dire al cuoco di accendere i fornelli, al sommelier di stappare il vino e al

lavapiatti di preparare i piatti puliti, impazziresti (questo è il **Problema** della slide: troppe classi, troppa complessità). La **Soluzione**? Il ristorante ti mette a disposizione una "Facciata": il **Cameriere**. Tu parli solo con lui ("Vorrei il menu numero 1"), e sarà lui a smazzarsi tutte le comunicazioni complesse con la cucina.

Lo scopo quindi è quello di evitare di chiamare 10 mila classi per svolgere un'azione, ma fare una classe di altissimo livello che raggruppi tutti questo in metodi semplicissimi. Il client parla solo con il facade.

COME RICONOSCERLO:

Il facade si riconosce perché nella classe del facade si dichiarano tantissime variabili, o fa chiamate di altre classi.

CODICE:

Qui ci sono le classi iniziali, quindi quelle che poi il facade raggrupperà per far chiamare al client un unico metodo che comprenda tutte le classi e le sue utenze.

```
Java

class ServizioMagazzino {
    public boolean verificaDisponibilita(String idProdotto) {
        System.out.println("🍷 [MAGAZZINO] Controllo disponibilità per l'articolo: " + idProdotto);
        return true; // Immaginiamo che ci sia
    }

    public void aggiornaGiacenza(String idProdotto) {
        System.out.println("🍷 [MAGAZZINO] Sottratto 1 pezzo dal database inventario.");
    }
}

class ServizioPagamenti {
    public boolean processaPagamento(String utente, double prezzo) {
        System.out.println("💳 [PAGAMENTI] Addebito di " + prezzo + "€ sul conto di " + utente);
        return true; // Immaginiamo che la carta sia valida
    }
}

class ServizioSpedizioni {
    public void organizzaConsegna(String idProdotto, String indirizzo) {
        System.out.println("📦 [SPEDIZIONI] Lettera di vettura creata per: " + indirizzo);
    }
}
```

Qui invece abbiamo il facade, dove crea le istanze per ogni classe di quelle fatte sopra, e poi svolge delle funzioni che il client chiamerà, e queste funzioni (in questo caso solo una completa acquisto) vanno a richiamare le istanze delle classi che crea sempre il facade.

Java

```

class OrdineFacade {
    private ServizioMagazzino magazzino;
    private ServizioPagamenti pagamenti;
    private ServizioSpedizioni spedizioni;

    public OrdineFacade() {
        // Il Facade inizializza e prepara tutti i sottosistemi complessi
        this.magazzino = new ServizioMagazzino();
        this.pagamenti = new ServizioPagamenti();
        this.spedizioni = new ServizioSpedizioni();
    }

    // IL METODO MAGICO: L'imbuto che nasconde la complessità
    public boolean completaAcquisto(String idProdotto, String utente, String indirizzo, c
        System.out.println("\n📦 Inizio elaborazione ordine per " + utente + "...");

        // Il Facade orchestra le operazioni nel giusto ordine
        if (!magazzino.verificaDisponibilita(idProdotto)) {
            System.out.println("❌ Errore: Prodotto esaurito.");
            return false;
        }

        if (!pagamenti.processaPagamento(utente, prezzo)) {
            System.out.println("❌ Errore: Pagamento rifiutato.");
            return false;
        }

        magazzino.aggiornaGiacenza(idProdotto);
        spedizioni.organizzaConsegna(idProdotto, indirizzo);
    }
}

```

Successivamente il client chiamerà la funzione con un istanza del facade, così che tutto quello che c'è nella funzione il client, lo chiama in un colpo solo.

Java

```

public class MainECommerce {
    public static void main(String[] args) {

        // 1. Istanziamo il nostro Facade
        OrdineFacade gestoreOrdini = new OrdineFacade();

        // 2. L'utente preme il tasto "Compra"
        // Il Client chiama un solo metodo, passandogli i dati di base.
        gestoreOrdini.completaAcquisto(
            "PC-GAMING-01",
            "Simone",
            "Via Roma 123, Catania",
            1250.50
        );
    }
}

```

BRIDGE->

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

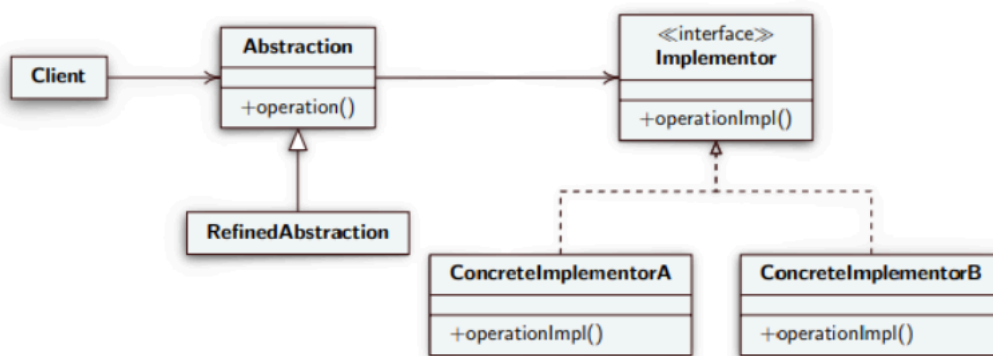
Separa "cos'è" un veicolo da "come funziona" il suo motore, creando due binari paralleli collegati da un ponte. Evita l'esplosione delle sottoclassi (es. invece di creare 16 classi per auto e motori, ne crei 4+4).

SCOPO:

Il pattern Bridge serve a **separare un'astrazione dalla sua implementazione**, permettendo a entrambe di evolvere e di essere modificate in modo totalmente indipendente. In parole semplici: serve a evitare che il numero di classi si moltiplichi all'infinito quando hai un oggetto che può avere diverse "forme" e diversi "comportamenti" (come abbiamo visto nell'esempio delle figure geometriche e dei colori).

COME RICONOSCERLO:

Lo riconosci a colpo d'occhio nel codice perché **ci sono due gerarchie di classi distinte, collegate da una variabile**. La classe principale (l'Astrazione) è di solito una classe astratta che **non implementa l'interfaccia**, ma la *contiene* come parametro (il ponte). Quando chiami un metodo sull'Astrazione, lei non fa il lavoro sporco, ma delega l'operazione all'oggetto Implementazione che le hai "agganciato" dentro.



-Abstraction: L'astrazione che è la nostra classe astratta "Forma" in questo caso, quindi è la classe generica di base di quello che è il tuo oggetto. Dentro questa classe c'è la funzione "Disegna" per fare la forma, ma al suo interno c'è anche il PONTE, si nota guardando la freccia che va da abstraction a implementor, e questo ponte dice che nella nostra classe c'è una variabile di tipo colore;

-RefinedAbstraction: Sono le classi figlie della nostra astrazione che semplicemente fanno la forma vera e propria;

-Implementor: L'interfaccia caratteristica che hai deciso di staccare, in modo tale che si possa creare il ponte, e al suo interno vi deve essere un metodo come "applicaTinta()".

-ConcreteImplementor A e B: Sono i colori veri e propri e fanno il lavoro pratico di colorare le forme.

VANTAGGI:

- **Basta esplosione di classi:** Invece di avere 16 classi incrociate, ne avrai solo 4 da una parte e 4 dall'altra.
- **Flessibilità totale:** Puoi prendere un "pezzo" di destra e agganciarlo a un "pezzo" di sinistra a runtime, mentre il programma sta girando.
- **Codice pulito e aperto alle estensioni:** Se devi aggiungere una nuova variante (es. un nuovo motore), crei solo una piccola classe senza dover mai toccare il codice dei veicoli esistenti.

ESEMPIO CODICE IN CUI USARE IL BRIDGE:

L'esempio pratico: Immagina di avere la classe base `Forma`. Da questa crei le classi figlie `Cerchio` e `Rettangolo`. Fin qui tutto ok. Ora il capo ti dice: "Le forme devono avere un colore: Rosso o Blu".

- Senza il pattern Bridge, tu usi l'ereditarietà e crei: `CerchioRosso`, `CerchioBlu`, `RettangoloRosso` e `RettangoloBlu`. (Totale: 4 classi).
- Ora il capo vuole aggiungere il `Triangolo`. Devi creare `TriangoloRosso` e `TriangoloBlu`. (Totale: 6 classi).
- Ora il capo vuole aggiungere il colore `Verde`. Devi creare `CerchioVerde`, `RettangoloVerde`, `TriangoloVerde`. (Totale: 9 classi).
- Capisci il problema? **Il numero di classi si moltiplica a dismisura!** Se hai 10 forme e 10 colori, devi scrivere 100 classi diverse!

La soluzione sarebbe quella di dividere l'astrazione dall'implementazione, costruendo un ponte(bridge), dove la classe astratta `forma` contiene `quadrato`, `rettangolo` ecc.. e poi l'implementazione contiene le sue caratteristiche, così che si crea una forma, e poi si ha il ponte che lo fa puntare al colore.

CODICE:

```
public class Client {  
    public static void main(String[] args) {  
        Forma rettangoloBlu = new Rettangolo("Blu");  
        rettangoloBlu.disegna();  
        Forma cerchioRosso = new Cerchio("Rosso");  
        cerchioRosso.disegna();  
    }  
}
```

```
public abstract class Forma {  
    public abstract void disegna();  
    protected Disegno disegno;  
}
```

```
public class Rettangolo extends Forma {  
  
    public Rettangolo(String colore){  
        if(colore.equals("Rosso"))  
            this.disegno = new DisegnoRosso();  
        else if(colore.equals("Blu"))  
            this.disegno = new DisegnoBlu();  
    }  
  
    public void disegna() {  
        disegno.disegnaRettangolo();  
    }  
}
```

```

public class Cerchio extends Forma {

    public Cerchio(String colore){
        if(colore.equals("Rosso"))
            this.disegno = new DisegnoRosso();
        else if(colore.equals("Blu"))
            this.disegno = new DisegnoBlu();
        }

    public void disegna() {
        disegno.disegnaCerchio();
    }
}

```

```

public interface Disegno {
    void disegnaRettangolo();
    void disegnaCerchio();
}

```

```

public class DisegnoBlu implements Disegno {

    public void disegnaRettangolo() {
        System.out.println("Rettangolo blu.");
    }

    public void disegnaCerchio() {
        System.out.println("Cerchio blu.");
    }
}

```

```

public class DisegnoRosso implements Disegno {

    public void disegnaRettangolo() {
        System.out.println("Rettangolo rosso.");
    }

    public void disegnaCerchio() {
        System.out.println("Cerchio rosso.");
    }
}

```

COMPOSITE->

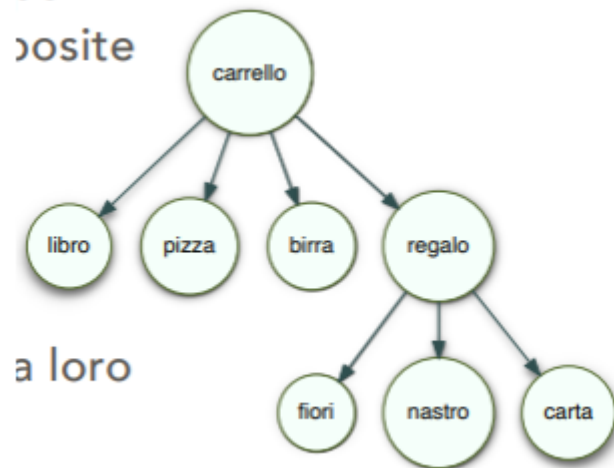
BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Tratta i singoli oggetti e i gruppi di oggetti (strutture ad albero ricorsive) esattamente nello stesso modo. Il Client non fa differenze (es. calcolare il prezzo di una singola mela o di un intero cesto regalo).

SCOPO:

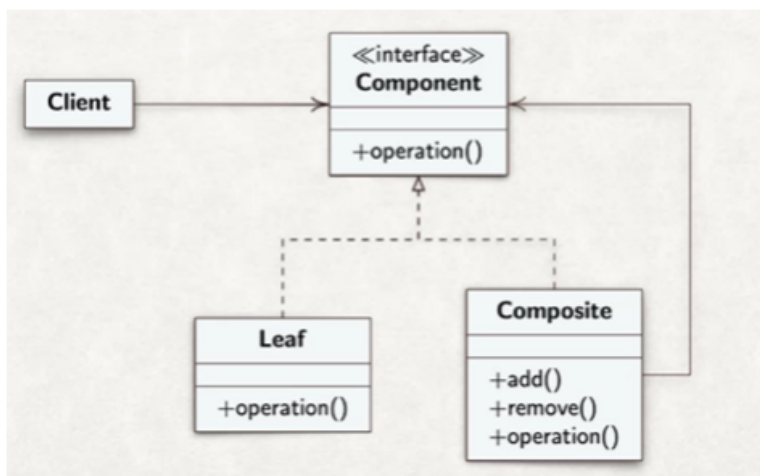
Il pattern composite è un pattern che riesce a trattare oggetti composti e oggetti singoli in

modo uniforme. Tale pattern usa una struttura ad albero per rappresentare gli oggetti. Quindi la cosa importante di questo pattern è che lui tratta allo stesso modo un elemento singolo, e un gruppo di elementi. Quindi quando si rappresenta un oggetto complesso, con unione di oggetti semplici, è sconsigliato fare 2 metodi diversi per gestirli entrambi in solitaria. La cosa ideale è quella di trattare gli oggetti composti in modo ricorsivo, quindi grazie all'albero si usa la struttura ricorsiva comunicando direttamente con la radice.



Guardando questo diagramma vediamo come tutti gli oggetti vengono trattati allo stesso modo. Vediamo il carrello che è la radice, che è un oggetto complesso che a sua volta è composto da: libro pizza birra e il regalo. Quest'ultimo è composto da altri elementi più semplici. Quando il client andrà a chiedere il prezzo totale della spesa al carrello, il carrello gli restituirà diretto il prezzo finale, non avrà bisogno di chiederlo a tutti, e il regalo calcolerà il suo prezzo totale tra lui e lui.

COME RICONOSCERLO:



Component: Il component è il padre di tutti gli elementi, e stabilisce la regola fondamentale che tutti devono rispettare. Possiede una funzione che tutti devono rispettare come vediamo nel diagramma UML.

Leaf: È una sottoclasse di component, e implementa component dato che è la classe degli elementi semplici. Dato che eredita le funzioni da component, lei avrà la funzione operation() ma al suo interno non verrà scritto niente dato che è un oggetto semplice;

Composite: Il composite invece è la sottoclasse di component ma degli oggetti composti, eredita l'operation() del component, ma al suo interno nella classe ha altri 2 metodi che sono

add e remove che lo rendono composite. Vediamo che c'è anche una freccia ricorsiva che va da composite verso component, che indica la ricorsione per calcolare gli oggetti composti.

Client: Il client se vediamo nel diagramma UML parla solo con il component, quindi non gli interessa se un oggetto sia leaf o composite, gli interessa solo il prezzo.

CODICE:

```
import java.util.ArrayList;
import java.util.List;

// =====
// 1. COMPONENT (L'interfaccia base)
// Definisce la regola d'oro: tutto ciò che va nel carrello deve avere un prezzo.
// =====
interface ComponenteCarrello {
    double getPrezzo();
    void stampaDettagli(String spazio); // Serve solo per stampare lo scontrino bello da
}

// =====
// 2. LEAF (L'elemento semplice)
// Non contiene nient'altro. È la fine dell'albero.
// (Rappresenta il libro, la pizza, la birra, il nastro, i fiori)
// =====
class ProdottoSingolo implements ComponenteCarrello {
    private String nome;
    private double prezzo;

    public ProdottoSingolo(String nome, double prezzo) {
        this.nome = nome;
        this.prezzo = prezzo;
    }

    @Override
    public double getPrezzo() {
        return this.prezzo;
    }
}
```

```
// =====
// 3. COMPOSITE (L'elemento composto / Contenitore)
// È esso stesso un ComponenteCarrello, MA contiene una lista di altri componenti!
// (Rappresenta il Carrello stesso, o il Regalo incartato)
// =====
class ContenitoreProdotti implements ComponenteCarrello {
    private String nomeContenitore;
    // ECCO LA FRECCIA RICORSIVA DELLA SLIDE:
    // Un Composite contiene una lista di Componenti generali!
    private List<ComponenteCarrello> elementiInterni = new ArrayList<>();

    public ContenitoreProdotti(String nome) {
        this.nomeContenitore = nome;
    }

    // Metodi specifici del Composite (per aggiungere/rimuovere i rami)
    public void aggiungi(ComponenteCarrello c) {
        elementiInterni.add(c);
    }

    // IL METODO MAGICO: Tratta i figli tutti allo stesso modo
    @Override
    public double getPrezzo() {
        double totale = 0;
        // Il contenitore non sa se sta sommando Prodotti Singoli o altri Contenitori.
        // Chiama semplicemente getPrezzo() su tutti, e la ricorsione fa il resto!
        for (ComponenteCarrello componente : elementiInterni) {
            totale += componente.getPrezzo();
        }
        return totale;
    }
}
```

Java



```
public class MainEcommerce {
    public static void main(String[] args) {

        // 1. Creiamo le "Foglie" sfuse (LEAF)
        ComponenteCarrello libro = new ProdottoSingolo("Libro 'Design Patterns'", 45.00);
        ComponenteCarrello pizza = new ProdottoSingolo("Pizza Margherita surgelata", 3.50);
        ComponenteCarrello birra = new ProdottoSingolo("Birra artigianale", 4.00);

        // 2. Creiamo il Regalo, che è un "Contenitore" (COMPOSITE)
        ContenitoreProdotti regalo = new ContenitoreProdotti("Regalo per Ciccio");
        regalo.aggiungi(new ProdottoSingolo("Fiori", 15.00));
        regalo.aggiungi(new ProdottoSingolo("Carta regalo lucida", 2.00));
        regalo.aggiungi(new ProdottoSingolo("Nastro rosso", 1.00));

        // 3. Creiamo il Carrello generale (COMPOSITE principale)
        ContenitoreProdotti carrello = new ContenitoreProdotti("Carrello Spesa di Simone");
        carrello.aggiungi(libro);
        carrello.aggiungi(pizza);
        carrello.aggiungi(birra);
        // INSERISCO UN CONTENITORE DENTRO UN ALTRO CONTENITORE!
        carrello.aggiungi(regalo);

        // 4. IL CLIENT IN AZIONE
        // Al Client basta chiamare un solo metodo.
        carrello.stampaDettagli("");
        System.out.println("\n👉 TOTALE DA PAGARE: " + carrello.getPrezzo() + "€");
    }
}
```

DECORATOR->

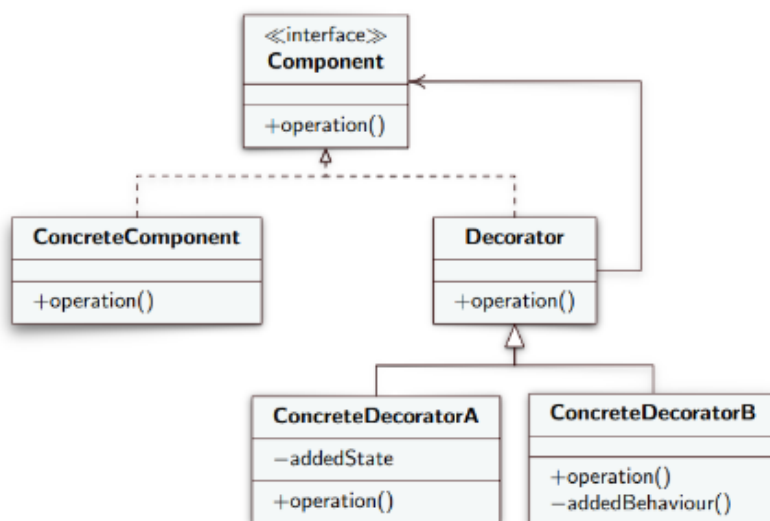
BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Il "vestito a cipolla". Permette di aggiungere nuove funzionalità a un *singolo specifico oggetto* a runtime, avvolgendolo dentro altre classi, senza usare la rigida ereditarietà (es. prendere un caffè base e aggiungergli panna e caramello, senza dover creare una classe fissa `CaffèPannaCaramello`). Quindi invece che alla classe caffè, si aggiunge l'ereditarietà aggiungi panna, lo si fa tramite il decorator.

SCOPO:

Il decorator ha un nome azzecato, perché funge da decoratore di oggetti a runtime. Immagina di avere una classe che ne estende un'altra, ma con la parolina magica `extends` in pratica modifichi i comportamenti degli oggetti futuri, e lo fai in modo rigido e definitivo. Tramite questo pattern invece puoi decidere di modificare semplicemente un'oggetto già creato, e aggiungergli super poteri al volo, lasciando gli altri della sua specie intatti. Quindi il decorator svolge una funzione che la rigida ereditarietà tra classi non può fare, perché quest'ultima modificherebbe tutta la sottoclasse e non solo un'istanza.

COME RICONOSCERLO:



Guardando il seguente diagramma UML vediamo come nello schema ci sono diversi elementi:

-Component: È l'interfaccia base, e quindi definisce cosa è l'oggetto nudo e crudo, come ad esempio un caffè. Al suo interno ha la solita operazione che fa capire che qualsiasi oggetto di tipo caffè deve avere quella operazione;

-ConcreteComponent: È l'oggetto reale di base, ossia quello che verrà decorato, quindi ad esempio un caffè normale;

-Decorator: Il decorator è la classe magica perché fa 2 cose diverse:

1-Punta verso l'alto e quindi implementando l'interfaccia component, e quindi agli occhi del sistema è un caffè;

2-Torna indietro con la freccia nera continua, quindi al suo interno contiene un caffè;

3-Il suo scopo quindi è quello di acchiappare le richieste esterne e di inoltrarle a quello che lui ha nella pancia, quindi il caffè;

-ConcreteDecorator(A e B): Questi 2 sono le decorazioni vere e proprie, quindi aggiuntaPanna o aggiuntaZucchero. Ereditano da decorator, e quindi dopo aver mandato la richiesta al decorator, la loro richiesta viene aggiunta dentro quello che c'è nel decorator. Queste decorazioni dovranno rimanere private e sconosciute al client, dato che dovrà solo chiamare l'oggetto caffè.

CODICE:

Come prima parte del codice implementiamo il component e il concrete component. Il component è l'interfaccia base che descrive le regole fisse per qualsiasi pezzo o veicolo intero, quindi hanno un costo e una descrizione. Il concreteComponent è il vero e proprio oggetto da decorare che eredita i metodi dal component.

```
// =====  
// 1. COMPONENT (L'interfaccia base)  
// La regola fissa per qualsiasi pezzo o veicolo intero  
// =====  
public interface ConfigurazioneKart {  
    String getDescrizione();  
    double getCosto();  
}  
  
// =====  
// 2. CONCRETE COMPONENT (L'oggetto nudo e crudo)  
// Il nostro elemento di partenza da cui tutto inizia  
// =====  
public class KartBase implements ConfigurazioneKart {  
    @Override  
    public String getDescrizione() {  
        return "Kart telaio standard";  
    }  
  
    @Override  
    public double getCosto() {  
        return 1500.00;  
    }  
}
```

Qui invece abbiamo il decorator che sarebbe la classe astratta che implementa l'interfaccia descritta dal component, ma contiene anche il component stesso da decorare.

```
// =====  
// 3. DECORATOR (La "scatola" astratta)  
// Implementa l'interfaccia, ma contiene anche un riferimento ad essa!  
// =====  
public abstract class AccessorioDecorator implements ConfigurazioneKart {  
    // Il riferimento all'oggetto interno (il Component)  
    protected ConfigurazioneKart kartDaDecorare;  
  
    // Il costruttore accetta un qualsiasi Component (KartBase o altri Decorator!)  
    public AccessorioDecorator(ConfigurazioneKart kart) {  
        this.kartDaDecorare = kart;  
    }  
  
    @Override  
    public String getDescrizione() {  
        return kartDaDecorare.getDescrizione(); // Inoltra la chiamata all'interno  
    }  
  
    @Override  
    public double getCosto() {  
        return kartDaDecorare.getCosto(); // Inoltra la chiamata all'interno  
    }  
}
```

Questi invece sono i CONCRETEDECORATOR, che sono classi vere e proprie che implementano la classe astratta decorator, che hanno gli stessi metodi della classe astratta,

e aggiungono decorazioni al nostro component che partiva da zero.

```
public class TelemetriaAvanzata extends AccessorioDecorator {

    public TelemetriaAvanzata(ConfigurazioneKart kart) {
        super(kart);
    }

    @Override
    public String getDescrizione() {
        // Chiama la stringa base e aggiunge il suo "pezzettino"
        return super.getDescrizione() + " + Sensori Telemetria";
    }

    @Override
    public double getCosto() {
        // Chiama il costo base e somma il suo prezzo
        return super.getCosto() + 250.00;
    }
}

public class GommeDaBagnato extends AccessorioDecorator {

    public GommeDaBagnato(ConfigurazioneKart kart) {
        super(kart);
    }

    @Override
    public String getDescrizione() {
        return super.getDescrizione() + " + Set Gomme Rain";
    }

    @Override
    public double getCosto() {
        return super.getCosto() + 180.00;
    }
}
```

E poi c'è il main, che ci descrive come usare queste classi.

```
public class MainOfficina {
    public static void main(String[] args) {

        // 1. Il cliente compra solo il kart nudo (ConcreteComponent)
        ConfigurazioneKart ordine1 = new KartBase();
        System.out.println(ordine1.getDescrizione() + " | Totale: " + ordine1.getCosto());

        System.out.println("-----");

        // 2. Il cliente vuole un kart "full optional".
        // Avvolgiamo il kart base dentro le gomme, e poi tutto dentro la telemetria!
        ConfigurazioneKart ordine2 = new TelemetriaAvanzata(new GommeDaBagnato(new KartBase()));

        System.out.println(ordine2.getDescrizione() + " | Totale: " + ordine2.getCosto());
    }
}
```

PATTERN COMPORTAMENTALI->

I pattern comportamentali non creano né montano nulla, ma decidono chi fa cosa e come i pezzi comunicano tra loro durante il programma. I pattern comportamentali quindi decidono le responsabilità di ognuno degli oggetti. In un programma ci sono una sfilza di oggetti che devono comunicare tra loro e devono svolgere il loro compito, ma se lasci che ognuno di loro chiami a casaccio i metodi degli altri allora si viene a formare quello che chiamiamo spaghetti code.

I pattern comportamentali svolgono questi compiti:

-Gestiscono le comunicazioni(centralino)-> Impediscono che tutti gli oggetti debbano conoscersi a vicenda. Ad esempio i piloti degli aerei non si chiamano al telefono tra di loro per decidere chi atterra prima (sarebbe un disastro). Tutti parlano *solo* con la Torre di Controllo, che smista i messaggi. Nel software, questo lo fa il pattern **Mediator**. Oppure, pensa a YouTube: tu ti iscrivi a un canale e, quando esce un video, ti arriva la notifica senza che tu debba controllare ogni minuto. Questo è l'**Observer**.

-**Dividono il Lavoro (Lo Scaricabarile e le Tattiche)**-> Decidono a chi tocca risolvere un determinato problema in modo flessibile .*Esempio reale*: Se chiami il servizio clienti di Amazon, parli prima col bot, poi con l'operatore di primo livello, poi col manager. La tua richiesta "viaggia" lungo una catena finché qualcuno non ha l'autorità per risolverla. Questo è il pattern **Chain of Responsibility**.

-**Gestiscono gli "Stati d'animo" (Reazioni dinamiche)**-> Permettono a un oggetto di reagire in modo completamente diverso alla stessa identica richiesta, in base alla situazione in cui si trova.

- *Esempio reale*: Prendi il tasto "Seleziona Caffè" di un distributore automatico. Se premi il tasto quando non hai messo i soldi, la macchinetta fa un *Bip* di errore. Se lo premi dopo aver inserito 1 euro, ti fa il caffè. Il tasto è lo stesso, ma il suo comportamento è cambiato in base allo "stato" in cui si trovava la macchina. Questo è il pattern **State**.

In sintesi: **mettono ordine nel flusso di lavoro e nelle comunicazioni del tuo programma.**

STATE->

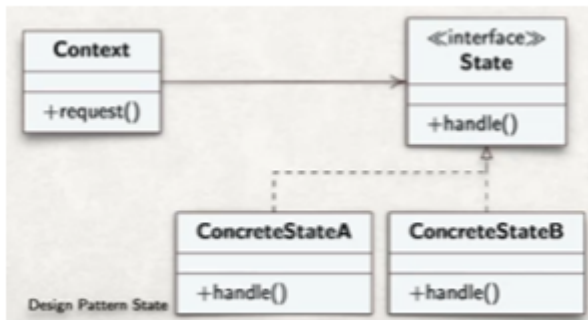
BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Il "Cambio di personalità". Permette a un oggetto di cambiare totalmente il suo comportamento quando cambia il suo stato interno. Agli occhi di chi lo usa, sembra che l'oggetto abbia cambiato classe (es. i tasti di un distributore automatico fanno cose diverse se hai inserito i soldi o se è vuoto).

SCOPO:

Lo scopo del pattern state(stato) permette di cambiare il comportamento dell'oggetto in base al momento e alla situazione in cui si trova, senza cambiare l'oggetto stesso. Se si pensa al tasto quadrato del joystick su fifa 19, tale tasto cambia in base alla situazione in cui si trova. Se stiamo difendendo quel tasto funge da contrasto, se stiamo attaccando quel tasto funge da tiro. Lo scopo del pattern quindi è togliere tutti quegli if che invadono il nostro programma, quindi se ad esempio per decidere cosa quel tasto deve fare mettiamo tanti if, tale pattern decide di fare tante piccole classi separate, una per ogni stato.

COME RICONOSCERLO:



-Context: Il context è l'oggetto principale con cui interagisce l'utente, in questo caso la partita. Il context al suo interno possiede lo stato attuale, e quando gli chiedi di fare qualcosa, passa la palla allo stato corrente;

-State: Lo state è l'interfaccia che contiene la regola base, ossia che obbliga tutti gli stati ad avere gli stessi metodi, come ad esempio `premiQuadrato`;

-ConcreteState(A e B): Sono le singole classi che rappresentano i vari stati in cui ci si può trovare a cliccare quel determinato tasto, quindi attacca o difendi.

Il vantaggio importante di questo pattern, è che se in futuro vorrai aggiungere una nuova funzionalità, basterà aggiungere una nuova classe.

CODICE:

```
// =====  
// 1. STATE (L'Interfaccia)  
// Definisce le azioni che si possono fare, indipendentemente dallo stato  
// =====  
public interface StatoPartita {  
    // Passiamo il Context al metodo così lo Stato può dirgli di cambiare!  
    void premiTastoStart(Partita contesto);  
}  
  
// =====  
// 2. CONCRETE STATES (I comportamenti specifici)  
// =====  
public class StatoMenu implements StatoPartita {  
    @Override  
    public void premiTastoStart(Partita contesto) {  
        System.out.println("🚩 Fischio d'inizio! I giocatori scendono in campo.");  
        // Cambio lo stato della partita!  
        contesto.setStato(new StatoInGioco());  
    }  
}  
  
public class StatoInGioco implements StatoPartita {  
    @Override  
    public void premiTastoStart(Partita contesto) {  
        System.out.println("⏸️ Gioco in pausa. Apertura del menu formazioni e tattiche.");  
        // Cambio lo stato della partita!  
        contesto.setStato(new StatoPausa());  
    }  
}  
  
public class StatoPausa implements StatoPartita {  
    @Override  
    public void premiTastoStart(Partita contesto) {  
        System.out.println("▶️ Ripresa del gioco. Palla al centro!");  
        // Ritorno in gioco  
        contesto.setStato(new StatoInGioco());  
    }  
}
```

```
// =====
// 3. CONTEXT (La Partita)
// È l'oggetto che usa il Client. Non ha "if", si fida del suo stato attuale.
// =====
public class Partita {
    // Tiene traccia dello stato in cui si trova in questo momento
    private StatoPartita statoCorrente;

    public Partita() {
        // Quando creo la partita, parte dal Menu
        this.statoCorrente = new StatoMenu();
    }

    public void setStato(StatoPartita nuovoStato) {
        this.statoCorrente = nuovoStato;
    }

    // L'utente preme un tasto sul controller.
    // La partita non fa calcoli, delega tutto allo stato attuale!
    public void premiStart() {
        statoCorrente.premiTastoStart(this);
    }
}
}
```

Java

```
public class MainTorneo {
    public static void main(String[] args) {

        Partita finale = new Partita();

        // 1. Siamo nel menu, l'utente preme start per iniziare
        finale.premiStart();

        // 2. Il gioco è in corso, l'utente preme start per mettere in pausa
        finale.premiStart();

        // 3. Il gioco è in pausa, l'utente preme start per riprendere
        finale.premiStart();
    }
}
```

OBSERVER->

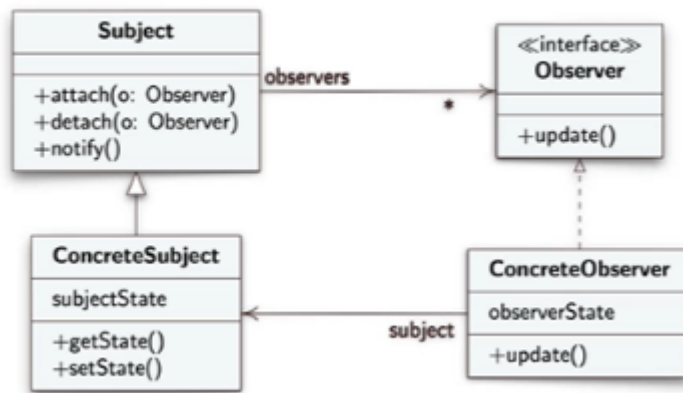
BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Il pattern "Iscriviti e attiva la campanellina" (Publish-Subscribe). Un oggetto centrale tiene una lista di iscritti. Appena il suo stato cambia, avvisa tutti automaticamente (es. il mercato che avvisa te e i tuoi amici quando il prezzo di un giocatore crolla).

SCOPO:

È il pattern più utilizzato nell'informatica moderna, ad esempio dai bot di telegram, o dalle notifiche di youtube. Questo perché si divide in una relazione uno-a-molti. Un vip(il singolo) è ad esempio uno youtuber, e gli iscritti sono i molti. Quando lui deve comunicare qualcosa, non va a cercare ad uno ad uno i suoi iscritti, ma semplicemente lancia un messaggio nel vuoto, che tutti i suoi iscritti leggeranno. Se il singolo VIP cambia stato, allora tutti i suoi iscritti saranno aggiornati e notificati.

COME RICONOSCERLO:



-Guardando questo diagramma UML, vediamo come l'observer è semplice da riconoscere, perché la classe principale (Subject) ha sempre 3 metodi inconfondibili che vediamo nell'immagine:

- `attach(Observer o)` : Il tasto "Iscriviti". Aggiunge un osservatore alla lista.
- `detach(Observer o)` : Il tasto "Annulla iscrizione".
- `notify()` : Il megafono. È il metodo che fa scattare l'avviso a tutti quelli presenti nella lista. Chiama il metodo `update` di ogni `observer`.

Il subject ha tanti observer che dipendono da esso, e che vengono informati per qualsiasi suo cambiamento.

-Observer: Gli observer sono interfacce che implementano il subject, e contengono il metodo `update()` che è la reazione che avranno quando il subject apporterà una modifica;

-ConcreteObserver: È chi riceve la notifica. Quando viene svegliato dal subject esegue il suo metodo `update()` che fa scattare la business logic, ovvero la reazione pratica in base alla comunicazione del subject. Ci sono 2 varianti dell'observer che permettono di far conoscere lo stato del concrete subject, e quindi di capire quale è la novità:

-Push observer: Il push observer imbocca il nuovo dato direttamente nella notifica, e quindi nel metodo `update` dell'observer stesso. Quindi il metodo diventa `update(nuovo dato)`. Questo tipo di observer è ottimo quando i dati da inserire nella notifica sono semplici e coincisi.

-Pull observer: Si usa quando il subject è pigro, quindi ti lancia semplicemente la notifica vuota senza dirti cosa è cambiato. Se interessato l'observer andrà a guardarsi la modifica e la inserirà nel metodo `update`.

-ConcreteSubject: Lui possiede i dati reali, e ogni volta che cambia uno di questi dati o il suo stato, deve attivare il metodo `notify()` per avvisare tutti gli observer.

VANTAGGI:

- **Disaccoppiamento totale (Applicabilità):** La slide dice che *"un oggetto deve notificare agli altri cose senza però doversi preoccupare su chi sono tali oggetti"*. Questo è il superpotere dell'Observer. Il canale YouTube non sa se tu sei un telefono, un PC, un tablet o una Smart TV. Non sa il tuo nome. Sa solo che sei nella lista e ti spara la notifica.
- **Salva il "Single Responsibility Principle":** Senza questo pattern, la classe principale dovrebbe avere chilometri di codice per calcolare cosa deve fare ogni singolo componente quando lei cambia. Con l'Observer, la classe principale pensa solo a se stessa; sono gli Observer che decidono come reagire alla notizia.

Ricordiamo che i subject e gli observed sono totalmente indipendenti tra loro. Nei vip è permesso mettere dei filtri sotto al quale avvisare gli observed, quindi se ad esempio un giocatore scende sotto ad un determinato prezzo avvisami, altrimenti no.

CODICE:

```
// =====
// 1. LE INTERFACCE (Le regole del gioco)
// =====

// Chi ascolta la notizia (Observer)
interface AllenatoreObserver {
    void update(String nomeGiocatore, int nuovoPrezzo);
}

// Chi pubblica la notizia (Subject)
interface MercatoSubject {
    void iscrivi(AllenatoreObserver o);
    void disiscriviti(AllenatoreObserver o);
    void notificaTutti();
}

// =====
// 2. CONCRETE SUBJECT (La carta sul mercato)
// =====
class CartaFifa implements MercatoSubject {
    private String nome;
    private int prezzoAttuale;

    // Il megafono: la lista di chi vuole la notifica!
    private List<AllenatoreObserver> iscritti = new ArrayList<>();

    public CartaFifa(String nome, int prezzoIniziale) {
        this.nome = nome;
        this.prezzoAttuale = prezzoIniziale;
    }
}
```

```

// Metodi per gestire le iscrizioni (attach / detach)
@Override
public void iscrivi(AllenatoreObserver o) { iscritti.add(o); }

@Override
public void disiscrivi(AllenatoreObserver o) { iscritti.remove(o); }

// Il metodo notify(): Scorri la lista e avvisa tutti
@Override
public void notificaTutti() {
    for (AllenatoreObserver iscritto : iscritti) {
        iscritto.update(this.nome, this.prezzoAttuale);
    }
}

// Quando il mercato aggiorna il prezzo, parte l'avviso automatico!
public void setPrezzo(int nuovoPrezzo) {
    System.out.println("\n [SERVER FIFA] Il prezzo di " + this.nome + " è variato a " + nuovoPrezzo);
    this.prezzoAttuale = nuovoPrezzo;
    notificaTutti(); // BOOM! Partono le notifiche
}

// =====
// 3. CONCRETE OBSERVER (I videogiocatori)
// =====
class GiocatoreFifa implements AllenatoreObserver {
    private String username;
    private int budget;

    public GiocatoreFifa(String username, int budget) {
        this.username = username;
        this.budget = budget;
    }

    // La reazione alla notifica!
    @Override
    public void update(String nomeGiocatore, int nuovoPrezzo) {
        if (nuovoPrezzo <= budget) {
            System.out.println(" [G] Notifica per " + username + ": " + nomeGiocatore + " è variato a " + nuovoPrezzo);
        } else {
            System.out.println(" [G] Notifica per " + username + ": " + nomeGiocatore + " è variato a " + nuovoPrezzo + " ma il budget è " + budget);
        }
    }
}

Java

public class MercatoMain {
    public static void main(String[] args) {

        // 1. Creiamo il VIP (L'oggetto da osservare)
        CartaFifa mbappe = new CartaFifa("K. Mbappé", 1500000);

        // 2. Creiamo i fan (Gli Observer)
        GiocatoreFifa simone = new GiocatoreFifa("SimoneGamer", 1000000); // Ha 1 milione di budget
        GiocatoreFifa carmelo = new GiocatoreFifa("CarmeloPro", 800000); // Ha 800k di budget

        // 3. I fan si iscrivono alle notifiche per quella carta (attach)
        mbappe.iscrivi(simone);
        mbappe.iscrivi(carmelo);

        // 4. Il server abbassa il prezzo. Le notifiche partono in automatico!
        mbappe.setPrezzo(1200000);

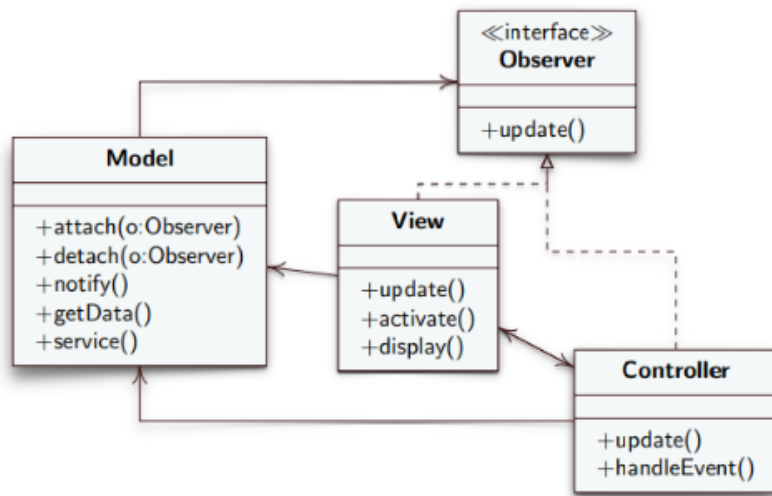
        // Il server crolla ancora di più!
        mbappe.setPrezzo(950000);
    }
}

```

MVC:

L'MVC è l'evoluzione del pattern observed, ed è l'architettura sulla quale si basa il 90% delle

applicazioni interattive e siti web.



-Model-> Il model gestisce le funzionalità e non dipende dall'output. È il subject dell'observed, ed è qui che risiede la logica pura. Non sa se l'utente sta usando un telefono o ilpc, ma lui appena cambia un suo dato avvisa subito tutti con la notifica;

-View-> La view sarebbe il nostro schermo, e quindi la parte grafica dell'observed. QUando il model con una notifica avvvisa che un suo dato è cambiato, dato che la view possiamo interpretarla come l'observed, aggiorna il tutto tramite l'update(), e aggiornando a schermo tale modifica;

-Controller-> Il controller è collegato all'utente, ed è la parte mancante dell'observed. È quello che informa il subject di ogni mossa dell'utente. Se ad esempio l'utente ha fatto un acquisto, il controller è quello che va ad avvisare il model e gli dice di scalargli i soldi dal conto.

MEDIATOR

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

La "Torre di controllo". Impedisce a una marea di oggetti di parlare tutti tra di loro (creando il caos). Gli oggetti parlano *solo* con il Mediatore, che smista i messaggi (es. gli aerei in un aeroporto non comunicano tra loro, ma solo con la torre).

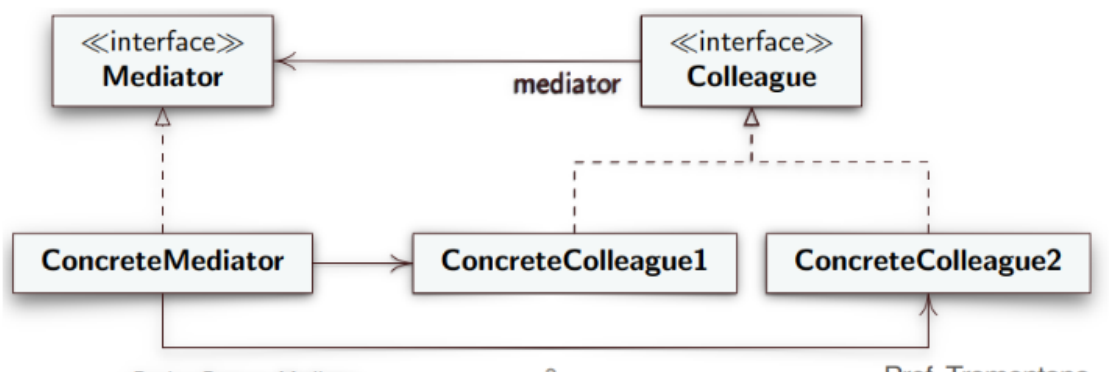
SCOPO:

Il mediator, dalla parola stessa mediatore, è colui che fa da ponte per le comunicazioni tra le varie classi, per evitare che il programma si trasformi in un vero spaghetti code tutto incasinato. Più classi ci sono e più è importante utilizzare il mediator. Il mediator non è altro quindi che un oggetto centrale che si occupa di far comunicare oggetti tra loro smistando messaggi. Ad esempio la torre di controllo di un aeroporto dice quale aereo deve atterrare o decollare, non sono gli aerei che comunicano tra loro.

COME RICONOSCERLO:

All'interno di tale design pattern vi è una classe che fa da mediator(mediatore) per tutte le altre, e bisogna guardare quella per riconoscere che è un mediator. Avere una classe che gestisce tutte le comunicazioni tra le classi rende il codice molti più semplice, e grazie a

questo infatti va via lo spaghetti code causato da chiamate di classi tra di loro che formano garbugli incredibili, e arriva la programmazione giusta fornitaci dal mediator.



I protagonisti di tale design pattern sono:

-Mediator: Il mediator è la nostra interfaccia mediatrice, che fa comunicare tutte le altre classi del programma che in questo caso sono i colleague. Definisce un metodo di comunicazione per tutti i colleague, ad esempio `inviameMessaggio()`;

-Colleague: I colleague come già detto sono le varie classi che devono comunicare tra loro, e in questo caso lo faranno tramite il mediator. I colleague sanno della loro esistenza e di quella del mediator, ma non di quella degli altri colleague.

-ConcreteMediator: È il vero e proprio cervellone del pattern. Contiene tutta la lista di colleague che ci sono nel programma, e ha la logica per coordinarli. Appena riceve il segnale da uno, decide lui chi deve attivarsi. Implementa i metodi dell'interfaccia mediator, e contiene un riferimento per ogni concreteColleague;

-ConcreteColleague: Sono le singole classi che svolgono azioni nel programma, e se qualcuno deve comunicare con un'altra classe, alza la mano e si rivolge al mediator.

PS: La complessità di comunicazione è tutta riposta sul mediator, infatti rende le classi colleague più facili da modificare e riutilizzare. Eventuali errori non si ripercuotono sulle altre classi ma al più sul mediator, che infatti non è riutilizzabile.

CODICE:

```
// =====  
// 1. MEDIATOR (L'Interfaccia della Centralina)  
// =====  
interface CentralinaMediator {  
    // Il metodo universale con cui i componenti comunicano con la centralina  
    void notifica(ComponenteSmart mittente, String evento);  
}  
  
// =====  
// 2. COLLEAGUE (La classe base dei dispositivi)  
// Tutti i dispositivi devono avere un collegamento al Mediator  
// =====  
abstract class ComponenteSmart {  
    protected CentralinaMediator mediator;  
  
    public ComponenteSmart(CentralinaMediator mediator) {  
        this.mediator = mediator;  
    }  
}  
  
// =====  
// 3. CONCRETE COLLEAGUES (I dispositivi veri e propri)  
// NON si conoscono tra loro. Parlano solo col Mediator.  
// =====  
class SensoreMovimento extends ComponenteSmart {  
    public SensoreMovimento(CentralinaMediator m) { super(m); }  
  
    public void rilevaOstacolo() {  
        System.out.println("🚗 [SENSORE]: Rilevato movimento sospetto!");  
        // Il sensore NON chiama l'allarme. Avvisa solo il Mediator!  
        mediator.notifica(this, "MOVIMENTO_RILEVATO");  
    }  
}
```

Nella prima immagine notiamo semplicemente l'interfaccia del mediator, nella quale viene implementata semplicemente il metodo notifica con la quale i componenti comunicano con la centralina. Poi la classe astratta del colleague che implementa un collegamento al mediator per ognuno dei concrete colleagues che successivamente imposteremo. E poi le classi dei concrete colleagues che sono i dispositivi veri e propri. Non si conoscono tra loro ma parlano solo con il mediator.

```

class AllarmeSonoro extends ComponenteSmart {
    public AllarmeSonoro(CentralinaMediator m) { super(m); }

    public void suona() {
        System.out.println("📢 [ALLARME]: WEE WEE WEE! Sirena attivata!");
    }
}

class LuciEsterne extends ComponenteSmart {
    public LuciEsterne(CentralinaMediator m) { super(m); }

    public void accendiTutto() {
        System.out.println("💡 [LUCI]: Fari di sicurezza accesi al 100%!");
    }
}

// =====
// 4. CONCRETE MEDIATOR (Il Cervellone)
// È l'unico a conoscere tutti i dispositivi e a coordinarli
// =====
class CentralinaDomotica implements CentralinaMediator {
    // La centralina tiene i riferimenti a tutti i Colleague
    private SensoreMovimento sensore;
    private AllarmeSonoro allarme;
    private LuciEsterne luci;

    // Metodi per registrare i dispositivi
    public void setSensore(SensoreMovimento s) { this.sensore = s; }
    public void setAllarme(AllarmeSonoro a) { this.allarme = a; }
    public void setLuci(LuciEsterne l) { this.luci = l; }

    // LA MAGIA DEL MEDIATOR: Lo smistamento dei messaggi
    @Override
    public void notifica(ComponenteSmart mittente, String evento) {

        // Se a chiamare è stato il sensore e ha visto un movimento...
        if (mittente == sensore && evento.equals("MOVIMENTO_RILEVATO")) {
            System.out.println("🟡 [CENTRALINA]: Valuto l'evento del sensore... Attivo p...");
            // ...la centralina coordina gli altri!
            luci.accendiTutto();
            allarme.suona();
        }
    }
}

```

Qui continuano i concrete colleagues con altri vari elementi, dove tutti fanno il proprio lavoro, ma al momento sono indipendenti tra loro, comunicano solo con il mediator. Poi c'è il concrete mediator che adesso conosce tutti i colleagues e li controlla e coordina. Registra tutti i dispositivi con le loro istanze, e poi fa l'override della funzione dell'interfaccia mediator, dove unisce tutti i colleagues, e lo fa grazie al sensore che scatta, e con l'if se il sensore scatta si accendono le luci e suona l'allarme.

```

Java

public class SmartHomeMain {
    public static void main(String[] args) {

        // 1. Creiamo la nostra "Torre di controllo"
        CentralinaDomotica cervello = new CentralinaDomotica();

        // 2. Creiamo i dispositivi, passandogli la centralina a cui fare riferimento
        SensoreMovimento sensoreGiardino = new SensoreMovimento(cervello);
        AllarmeSonoro sirena = new AllarmeSonoro(cervello);
        LuciEsterne fari = new LuciEsterne(cervello);

        // 3. Registriamo i dispositivi nella centralina
        cervello.setSensore(sensoreGiardino);
        cervello.setAllarme(sirena);
        cervello.setLuci(fari);

        // --- SIMULAZIONE ---
        System.out.println("--- Inizio simulazione notturna ---");

        // Un ladro entra in giardino. Il sensore scatta.
        sensoreGiardino.rilevaOstacolo();
    }
}

```

Infine qui abbiamo il client dove creiamo i dispositivi e passandogli la centralina a cui devono fare riferimento, e poi attiviamo il tutto.

CHAIN OF RESPONSIBILITY

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

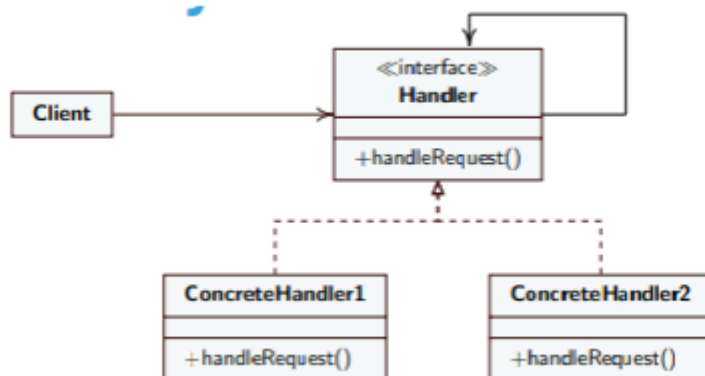
Lo "Scaricabarile" o "Passaparola". Crei una catena di oggetti. Se il primo non sa o non può gestire una richiesta, la passa al secondo, poi al terzo, finché qualcuno non la risolve (es. l'assistenza clienti: operatore di base -> tecnico -> manager).

SCOPO:

Il chain of responsibility, è un pattern che crea una catena di oggetti che permette di far disaccoppiare il mittente che fa una richiesta, dall'oggetto che soddisfa tale richiesta. Tale pattern può essere chiamato anche "Scaricabile", questo perché se un oggetto non sa soddisfare una richiesta, scarica la responsabilità all'oggetto successivo, come una catena, fino a che non ci sarà l'oggetto che soddisferà la richiesta. Immagina di essere in contatto con il tuo operatore telefonico perché non ti va internet. La tua richiesta è quella di risolvere tale problema:

- Risponde il bot automatico (non sa aiutarti) -> ti passa all'operatore di base.
- L'operatore di base (non ha i permessi) -> ti passa al tecnico specializzato.
- Il tecnico specializzato risolve il problema. Tu (il Client) hai fatto una sola richiesta, e la catena l'ha smistata automaticamente fino alla persona giusta.

COME RICONOSCERLO:



Guardando tale diagramma UML notiamo come c'è la nostra interfaccia Handler, che ha una freccia che ritorna su se stesso, quindi forma quello che è l'effetto ricorsivo. Questa è l'anima del pattern, perché se un oggetto non riesce a soddisfare una richiesta allora il programma torna sull'handler e chiede al prossimo oggetto. Nel diagramma UML vediamo:

-Handler: Contiene un metodo che è `handleRequest()`, ed è un'interfaccia dove al suo interno contiene un riferimento (variabile) al prossimo anello della catena, infatti lo vediamo tramite la freccia che ritorna su di esso. È il concetto astratto di anello della catena;

-Client: Il client è ovviamente quello che lancia la richiesta, ma non gli interessa chi gliela soddisfi, a lui basta che la soddisfi qualcuno;

-ConcreteHandler(1 e 2): Sono i veri e propri anelli della catena, che si prendono la responsabilità di decidere se possono o no risolvere la richiesta effettuata dal client, se la possono risolvere lo fanno, altrimenti la passano al suo successore. Dal diagramma vediamo come hanno una freccia puntata verso l'handler, il che significa che ereditano da lui le regole.

PS: Tale pattern è diverso dagli altri perché non sempre garantisce una soluzione, e quindi si deve mettere sempre un anello salvagente finale che ci dice che la richiesta è impossibile da soddisfare. Altra cosa è che anche a runtime posso aggiungere anelli nella catena.

CODICE:

```
// =====  
// 1. L'HANDLER ASTRATTO (La regola della catena)  
// =====  
abstract class SupportoHandler {  
    // ECCO LA FRECCIA DEL DIAGRAMMA UML!  
    // Ogni gestore sa chi è il prossimo a cui passare la patata bollente.  
    protected SupportoHandler successore;  
  
    // Metodo per agganciare l'anello successivo  
    public void setSuccessore(SupportoHandler successore) {  
        this.successore = successore;  
    }  
  
    // Il metodo che tutti devono implementare per gestire le richieste  
    public abstract void gestisciRichiesta(String livelloProblema, String descrizione);  
}  
  
// =====  
// 2. CONCRETE HANDLERS (Gli anelli della catena)  
// =====  
  
class OperatoreBase extends SupportoHandler {  
    @Override  
    public void gestisciRichiesta(String livelloProblema, String descrizione) {  
        if (livelloProblema.equals("SEMPLICE")) {  
            System.out.println("✅ [Operatore Base] Risolto: Ti ho resettato la password");  
        } else if (successore != null) {  
            System.out.println("👉 [Operatore Base] Problema troppo complesso. Passo al ");  
            successore.gestisciRichiesta(livelloProblema, descrizione);  
        }  
    }  
}  
  
class TecnicoSpecializzato extends SupportoHandler {  
    @Override  
    public void gestisciRichiesta(String livelloProblema, String descrizione) {  
        if (livelloProblema.equals("COMPLESSO")) {  
            System.out.println("🔧 [Tecnico] Risolto: Ho fixato il bug nel database. (" + descrizione + ")");  
        } else if (successore != null) {  
            System.out.println("👉 [Tecnico] Richiesta fuori dalle mie competenze. Passo ");  
            successore.gestisciRichiesta(livelloProblema, descrizione);  
        }  
    }  
}  
  
class Manager extends SupportoHandler {  
    @Override  
    public void gestisciRichiesta(String livelloProblema, String descrizione) {  
        if (livelloProblema.equals("CRITICO")) {  
            System.out.println("👑 [Manager] Risolto: Ho approvato il rimborso al cliente (" + descrizione + ")");  
        } else {  
            // Questo è l'ultimo anello della catena, il "salvagente"  
            System.out.println("❌ [Manager] Attenzione: Nessuno è in grado di gestire questa richiesta (" + descrizione + ")");  
        }  
    }  
}
```

In queste 2 prime immagini abbiamo l'handler astratto che è il gestore della catena che impone la regola per tutti gli anelli della catena, e sa chi è ogni singolo successore. Poi ci sono i concrete handlers che vediamo nel codice come le classi che estendono l'interfaccia base dell'handler, e tutte fanno un override della funzione che è all'interno dell'interfaccia. In ognuno di questi concreteHandlers, si controlla se sono in grado di soddisfare la richiesta, se lo sono la soddisfano, altrimenti la passano al successivo.

```

Java

public class HelpDeskMain {
    public static void main(String[] args) {

        // 1. Creiamo gli anelli (Gli oggetti)
        SupportoHandler livello1 = new OperatoreBase();
        SupportoHandler livello2 = new TecnicoSpecializzato();
        SupportoHandler livello3 = new Manager();

        // 2. Assembliamo la catena di responsabilità
        livello1.setSuccessore(livello2);
        livello2.setSuccessore(livello3);

        System.out.println("--- Richiesta 1: Reset Password ---");
        // Il Client parla SOLO col livello 1
        livello1.gestisciRichiesta("SEMPLICE", "L'utente ha dimenticato la password");

        System.out.println("\n--- Richiesta 2: Bug di sistema ---");
        // Il Client parla SOLO col livello 1
        livello1.gestisciRichiesta("COMPLESSO", "Errore 500 nel caricamento pagina");

        System.out.println("\n--- Richiesta 3: Rimborso Negato ---");
        // Il Client parla SOLO col livello 1
        livello1.gestisciRichiesta("CRITICO", "Cliente minaccia causa legale per disservizio");

        System.out.println("\n--- Richiesta 4: Extraterrestri ---");
        livello1.gestisciRichiesta("ALIENO", "UFO avvistato nel server");
    }
}

```

Qui nel client che da la richiesta, settiamo la nostra catena di responsabilità, e settiamo le varie richieste.

COMMAND

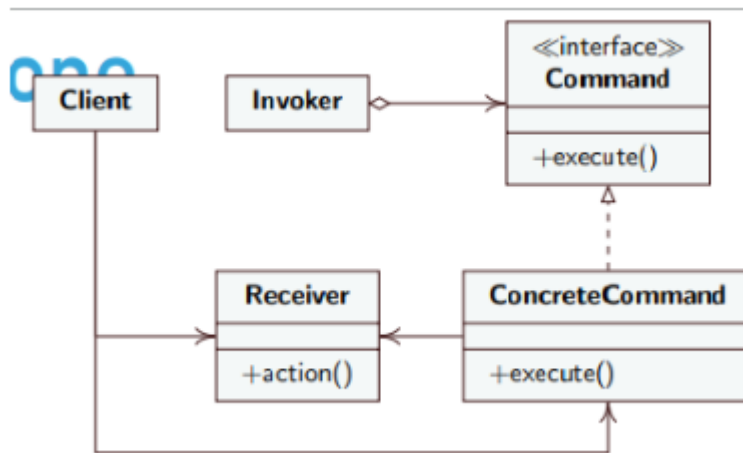
BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Il "Telecomando universale". Prende un'azione (o una richiesta) e la trasforma in un oggetto fisico. Questo ti permette di mettere le azioni in coda, programmarle per il futuro o annullarle (es. il tasto Ctrl+Z per annullare le mosse).

SCOPO:

Il command trasforma una richiesta in un oggetto fisico, quindi è come se prendessimo la richiesta fatta dal client e invece che svolgerla subito chiamando un metodo a lui associato, lo scriviamo in un foglietto di carta che poi sarà pronto per essere eseguito. Il command fa questo perché può mettere in coda questi oggetti che vengono creati per farli svolgere uno dopo l'altro al nostro programma, può creare un log, ossia che a fine programma si può ricapitolare quali siano state queste richieste, e poi c'è l'annullamento di tale richiesta prima che arrivi al programma.

COME RICONOSCERLO:



Il command come vediamo in questo diagramma UML ha tanti elementi che lo compongono, e lo possiamo riconoscere perché questi elementi svolgono specifici compiti, tra cui:

-Command: Il command è l'interfaccia astratta che rappresenta il biglietto vuoto su cui dovremo scrivere le varie azioni. Contiene dei metodi che tutti i comandi devono avere, tra cui **execute()** e magari anche metodi per essere annullati tali comandi;

-ConcreteCommand: Esso è l'ordine scritto sul foglio, e al suo interno ha il codice per chiamare il giusto receiver;

-Receiver: Il receiver può essere ad esempio il cuoco o il meccanico, che ha al suo interno la funziona **action()** che è utilizzata per svolgere l'azione e sa come cucinare la pizza o accendere il motore;

-Invoker: È chi prende il biglietto e innesca lo svolgimento dell'azione, funge da cameriere, perché lui non sa svolgere l'azione, ma sa innescare lo svolgimento(**execute()**);

-Client: Il client al solito siamo noi che decidiamo cosa ordinare(**concreteCommand**), da chi farlo fare(**Receiver**) e diamo il biglietto all'invoker.

CODICE:

```
// =====  
// 1. RECEIVER (Chi fa il lavoro sporco)  
// La meccanica pura. Non sa nulla di pulsanti o comandi.  
// =====  
class MotoreKart {  
    public void avvia() { System.out.println("🔥 [MOTORE] VROOOM! Motore avviato."); }  
    public void spegni() { System.out.println("🛑 [MOTORE] Motore spento."); }  
}  
  
// =====  
// 2. COMMAND (L'interfaccia del biglietto)  
// =====  
interface ComandoBox {  
    void execute(); // Esegui l'azione  
    void undo();    // Annulla l'azione (il famoso Ctrl+Z)  
}  
  
// =====  
// 3. CONCRETE COMMANDS (I singoli ordini incapsulati)  
// =====  
class ComandoAccensione implements ComandoBox {  
    private MotoreKart motore;  
  
    // Il comando viene "armato" collegandolo al ricevitore giusto  
    public ComandoAccensione(MotoreKart m) { this.motore = m; }  
  
    @Override  
    public void execute() { motore.avvia(); }  
  
    @Override  
    public void undo() { motore.spegni(); } // L'inverso dell'accensione  
}  
  
class ComandoSpegnimento implements ComandoBox {  
    private MotoreKart motore;  
  
    public ComandoSpegnimento(MotoreKart m) { this.motore = m; }  
  
    @Override  
    public void execute() { motore.spegni(); }  
  
    @Override  
    public void undo() { motore.avvia(); } // L'inverso dello spegnimento  
}  
  
// =====  
// 4. INVOKER (Il Pulsante sul volante)  
// Non sa cosa fa. Sa solo che se viene premuto, deve eseguire il comando che ha in pancia  
// =====  
class PulsanteVolante {  
    private ComandoBox comandoAssegnato;  
    private ComandoBox ultimoComandoEseguito; // Serve per l'annullamento!  
  
    // Assegniamo una funzione al pulsante  
    public void setComando(ComandoBox c) {  
        this.comandoAssegnato = c;  
    }  
  
    // Il pilota preme il tasto  
    public void premi() {  
        if (comandoAssegnato != null) {  
            comandoAssegnato.execute();  
            ultimoComandoEseguito = comandoAssegnato; // Mi segno cosa ho appena fatto  
        }  
    }  
  
    // Il pilota preme il tasto "Annulla"  
    public void premiAnnulla() {  
        if (ultimoComandoEseguito != null) {  
            System.out.println("🗑️ Annullamento ultima operazione...");  
        }  
    }  
}
```



```
// Il pilota preme il tasto "Annulla"
public void premiAnnulla() {
    if (ultimoComandoEseguito != null) {
        System.out.println("🚗 Annullamento ultima operazione...");
        ultimoComandoEseguito.undo();
    }
}
}
```

In queste 2 immagini notiamo come abbiamo il receiver che è la meccanica pura, quindi lui non sa niente di pulsanti. Poi abbiamo l'interfaccia del command dove ci sono solamente le azioni che i concrete commands implementano, ossia l'avvio e l'annullamento. Poi i concrete commands che implementano l'interfaccia del commands, con le 2 funzioni che sono per eseguire e annullare le azioni, e poi l'invoker che è il cameriere che innesca le azioni in base a cosa fa l'utente.

```
public class BoxGaraMain {
    public static void main(String[] args) {

        // 1. Creiamo la meccanica (Receiver)
        MotoreKart motoreVortex = new MotoreKart();

        // 2. Creiamo gli ordini incapsulati (ConcreteCommands)
        ComandoBox start = new ComandoAccensione(motoreVortex);
        ComandoBox stop = new ComandoSpegnimento(motoreVortex);

        // 3. Creiamo l'elettronica (Invoker)
        PulsanteVolante pulsanteRosso = new PulsanteVolante();

        System.out.println("--- SETUP NEL BOX ---");
        // Il meccanico decide che il pulsante rosso accenderà il motore
        pulsanteRosso.setComando(start);

        System.out.println("\n--- PILOTA IN PISTA ---");
        // Il pilota preme il pulsante. Il pulsante chiama l'execute.
        pulsanteRosso.premi();

        // Il pilota si accorge di aver sbagliato e preme annulla!
        pulsanteRosso.premiAnnulla();
    }
}
```

E poi il client.

ITERATOR

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

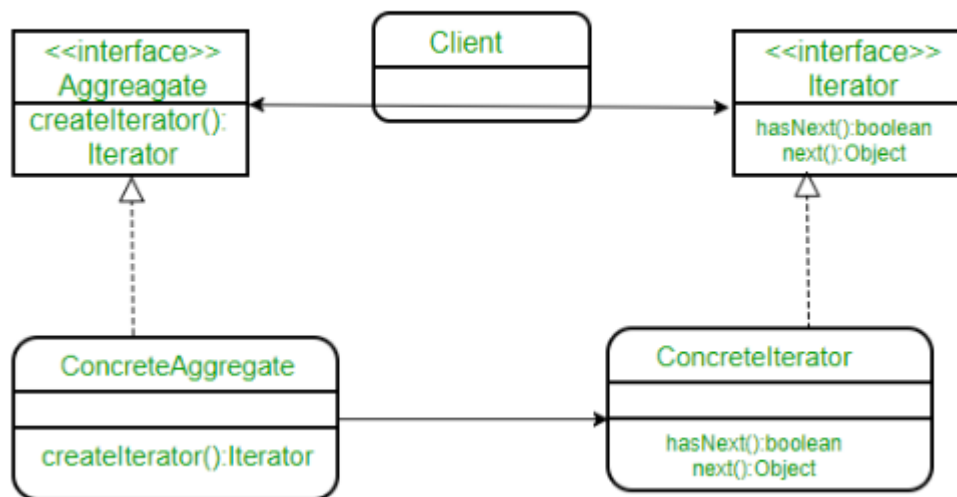
Lo "Scorritore". Permette di scorrere gli elementi di una collezione (come una lista o un albero) uno ad uno, senza che tu debba sapere come quella collezione è programmata internamente (es. quando usi un ciclo `for-each` in Java).

SCOPO:

Lo scopo dell'iterator, è quella di darti un telecomando in mano, permettendoti di scorrere ad uno ad uno gli elementi di una collezione o di un gruppo senza sapere come è formato all'interno tale gruppo. Quello che fa quindi è affidare lo scorrimento di tale struttura dati ad un iterator, invece che magari dividere la classe in tantissimi metodi inutili come `dammiPrimoElemento()`, `dammiProssimoElemento()` ecc.. . Quindi si toglie la responsabilità dell'accesso da un oggetto lista e lo si dà all'iterator. Questo oggetto è separato dalla lista, e fa da segnalibro per tale lista.

COME RICONOSCERLO:

L'iterator è un pattern utilissimo, dato che può scorrere gli elementi di tante strutture dati differenti. Immagina di avere una lista, una hash map e un albero. Tutte queste strutture dati memorizzano i dati in maniera differente, e senza l'iterator per scorrere tali dati si dovrebbero sapere 3 algoritmi a memoria e riempire il programma. Con l'iterator invece si mette in comune tutti, L'iterator ti fornisce un'interfaccia comune standardizzata (di solito con i famosi metodi `hasNext()` e `next()`). A te non interessa se sotto il cofano c'è un array o un albero: tu chiedi semplicemente *"C'è un prossimo elemento? Se sì, dammelo"*, e sarà l'iteratore specifico di quella struttura a farsi il mazzo per andarlo a ripescare nella memoria!



1)- Aggregate (L'Interfaccia della Libreria)

- **Nel diagramma:** Il rettangolo in alto a sinistra. Ha un solo metodo: `createIterator()`.
- **Cos'è:** È la regola base per qualsiasi oggetto che contenga più elementi (un "aggregato"). Dice semplicemente: *"Se sei una collezione di dati, devi fornirmi un metodo per ottenere un iteratore (un segnalibro) per scorrerti"*.

2)ConcreteAggregate (La tua Collezione reale)

- **Nel diagramma:** In basso a sinistra. Eredita da **Aggregate**.
- **Cos'è:** È il contenitore vero e proprio. Può essere un array, una lista, un albero. È lui che possiede fisicamente i dati salvati in memoria. Quando chiami il suo metodo `createIterator()`, lui fabbrica e ti restituisce il segnalibro adatto alla sua struttura.

3)Iterator (L'Interfaccia del Telecomando)

- **Nel diagramma:** In alto a destra. Ha i due famosi metodi `hasNext()` (C'è un prossimo elemento?) e `next()` (Dammi il prossimo elemento).
- **Cos'è:** È il "telecomando universale". Stabilisce le regole su come si scorre un gruppo di elementi. Non sa e non gli interessa cosa sta scorrendo.

4)ConcreteIterator (Il Segnalibro Fisico)

- **Nel diagramma:** In basso a destra. Implementa l'interfaccia `Iterator` ed è strettamente collegato al `ConcreteAggregate` (vedi la freccia che li unisce).
- **Cos'è:** È il motore vero e proprio. È un oggetto che tiene traccia della posizione attuale (`indice`). Conosce perfettamente come è fatto il `ConcreteAggregate` al suo interno, quindi sa esattamente come spostarsi da un elemento all'altro.

5)Client (Il Programma Principale)

- **Nel diagramma:** In alto al centro. Le frecce indicano che conosce *solo* le interfacce (`Aggregate` e `Iterator`).
- **Cos'è:** Sei tu. Nel tuo codice, tu chiedi alla collezione un iteratore, e poi usi il telecomando per scorrere i dati. Non vedi nulla di quello che succede nei "piani bassi".

CODICE:

```
// =====
// L'OGGETTO BASE
// =====
class Squadra {
    String nome;
    int overall;

    public Squadra(String nome, int overall) {
        this.nome = nome;
        this.overall = overall;
    }
}

// =====
// 1 e 3. LE INTERFACCE (Aggregate e Iterator del diagramma)
// =====
interface Campionato {
    IteratoreSquadre createIterator();
}

interface IteratoreSquadre {
    boolean hasNext();
    Squadra next();
}

// =====
// 2. CONCRETE AGGREGATE (La tua Lega Personalizzata)
// Contiene fisicamente l'elenco delle squadre (in questo caso in un array)
// =====
class LegaRoadToGlory implements Campionato {
    private Squadra[] squadre;
    private int indiceInserimento = 0;

    public LegaRoadToGlory() {
        // Creiamo una lega con un massimo di 10 squadre deboli
        squadre = new Squadra[10];
    }

    public void aggiungiSquadra(String nome, int overall) {
        if (indiceInserimento < 10) {
            squadre[indiceInserimento] = new Squadra(nome, overall);
            indiceInserimento++;
        }
    }

    // IL METODO CHIAVE: Fabbrica il segnalibro e glielo consegna!
    @Override
    public IteratoreSquadre createIterator() {
        return new SegnalibroLega(this.squadre);
    }
}
```

```
// =====
// 4. CONCRETE ITERATOR (Il Segnalibro / Telecomando)
// Sa come muoversi dentro un array. Tiene traccia della posizione!
// =====
class SegnalibroLega implements IteratoreSquadre {
    private Squadra[] squadreDaScorrere;
    private int posizioneCorrente = 0; // Il nostro segnalibro!

    public SegnalibroLega(Squadra[] squadre) {
        this.squadreDaScorrere = squadre;
    }

    @Override
    public boolean hasNext() {
        // C'è un prossimo elemento se non abbiamo superato l'ultimo slot pieno
        if (posizioneCorrente < squadreDaScorrere.length && squadreDaScorrere[posizioneCorrente] != null)
            return true;
        return false;
    }

    @Override
    public Squadra next() {
        // Prende la squadra attuale e avanza il segnalibro di 1 passo
        Squadra squadra = squadreDaScorrere[posizioneCorrente];
        posizioneCorrente++;
        return squadra;
    }
}
```

Java



```
public class FifaDraftMain {
    public static void main(String[] args) {

        // 1. Costruiamo il nostro ConcreteAggregate
        LegaRoadToGlory miaLega = new LegaRoadToGlory();
        miaLega.aggiungiSquadra("Pescara", 68);
        miaLega.aggiungiSquadra("Crotone", 70);
        miaLega.aggiungiSquadra("Benevento", 72);

        // 2. Chiediamo alla lega di darci il suo Iteratore (createIterator)
        IteratoreSquadre telecomando = miaLega.createIterator();

        System.out.println("🚀 SQUADRE SELEZIONATE PER LA LEGA 🚀\n");

        // 3. Scorriamo senza sapere che sotto c'è un Array! (hasNext e next)
        while (telecomando.hasNext()) {
            Squadra s = telecomando.next();
            System.out.println("- " + s.nome + " (Overall: " + s.overall + ")");
        }
    }
}
```

TEMPLATE METHOD

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

Lo "Scheletro della ricetta". Una classe madre scrive i passaggi fissi di un algoritmo, ma lascia vuoti dei "buchi". Le sottoclassi andranno a riempire solo quei buchi con i loro dettagli (es. la ricetta: bolli l'acqua -> *[aggiungi ingrediente]* -> versa in tazza. Le sottoclassi decidono se è tè o caffè).

SCOPO:

Il template method è un pattern molto utile ed efficiente, e si basa sul semplificare gli algoritmi e il codice per scriverli. Se abbiamo un algoritmo con 5 passaggi, dove l'1 il 3 e il 5

sono gli stessi passaggi per tutto il programma, invece il 2 e il 4 ogni classe figlia lo implementa per se. Questo si fa per evitare di riscrivere lo stesso codice ogni volta per ognuna delle classi figlie, e poi così ogni classe figlia può implementarsi i passaggi mancanti come vuole.

COME RICONOSCERLO:

Dobbiamo fare una distinzione tra librerie e framework:

-Librerie-> Le librerie nel programma sono cassette degli attrezzi, significa che il nostro codice richiama la libreria quando ha bisogno di fare un qualcosa, come ad esempio calcolare una radice quadrata;

-Framework-> Il framework funziona al contrario, ossia è una struttura fissa pre-costruita che richiama il nostro codice, nelle parti mancanti che ci sono al suo interno. Il template method è il re dei framework.

I buchi lasciati dal template method nel nostro algoritmo sono di due tipi:

-Metodi astratti-> I metodi astratti sono imposti dal padre ad essere scritti dai figli, significa che quel buco è completamente vuoto, e dato che il padre non lo sa fare lo affida al figlio, e se tale buco non verrà riempito ci sarà un errore di compilazione;

-Metodi Hook-> Sono passaggi *facoltativi*. La classe padre fornisce un metodo già fatto, ma vuoto (o con un comportamento di default). La classe figlia può decidere se ignorarlo oppure "agganciarci" del codice extra (override) per personalizzare l'algoritmo.

PS-> In tale pattern sappiamo che le classi figlie possono solo riempire buchi e non cambiare l'ordine dei passaggi dell'algoritmo. Altra cosa importante è che tale pattern è basato sul principio di hollywood, ossia che è la classe padre che chiamerà poi le classe figlie, e non viceversa.

CODICE:

Java



```
// =====  
// 1. LA CLASSE ASTRATTA (Il Framework / La Ricetta)  
// =====  
abstract class PreparazioneKart {  
  
    // IL TEMPLATE METHOD!  
    // Attenzione al 'final': le sottoclassi NON possono alterare l'ordine di questi passi  
    public final void preparaPerGara() {  
        mettiSulCavalletto();  
        montaGomme();           // Passaggio variabile (Astratto)  
        faiRifornimento();  
  
        if (vuoiTelemetria()) { // Passaggio variabile (Hook)  
            accendiSensori();  
        }  
  
        scendiDalCavalletto();  
        System.out.println("🏁 Kart pronto per la pista!\n");  
    }  
  
    // Passaggi INVARIANTI (Uguali per tutti, nascosti alle classi figlie)  
    private void mettiSulCavalletto() {  
        System.out.println("🏍️ 1. Kart alzato sul cavalletto.");  
    }  
    private void faiRifornimento() {  
        System.out.println("🏎️ 3. Miscela inserita nel serbatoio.");  
    }  
    private void scendiDalCavalletto() {  
        System.out.println("🏎️ 5. Kart calato a terra.");  
    }  
  
    // Passaggio VARIABILE OBBLIGATORIO (Abstract)  
    // Ogni tipo di setup deve specificare che gomme usare  
    protected abstract void montaGomme();  
  
    // Passaggio VARIABILE FACOLTATIVO (L'Hook!)  
    // Di default non montiamo la telemetria. Le figlie possono cambiare questa decisione  
    protected boolean vuoiTelemetria() {  
        return false;  
    }  
  
    // Metodo richiamato dall'Hook  
    private void accendiSensori() {  
        System.out.println("📡 4. Sensori di telemetria calibrati e attivi.");  
    }  
}  
  
// =====  
// 2. LE CLASSI CONCRETE (I setup specifici)  
// Riempono solo i "buchi" lasciati dal padre  
// =====  
  
class SetupQualificaAsciutto extends PreparazioneKart {  
  
    // Obbligato a implementarlo!  
    @Override  
    protected void montaGomme() {  
        System.out.println("🏎️ 2. Montate gomme Slick a mescola morbida (pressione 0.6 bar).");  
    }  
  
    // Sfrutta l'Hook: in qualifica la telemetria mi serve assolutamente!  
    @Override  
    protected boolean vuoiTelemetria() {  
        return true;  
    }  
}
```

```

class SetupGaraBagnato extends PreparazioneKart {

    // Obbligato a implementarlo!
    @Override
    protected void montaGomme() {
        System.out.println("🌀 2. Montate gomme Rain intagliate (pressione 1.0 bar).");
    }

    // Non fa l'override dell'Hook `vuoiTelemetria`.
    // Si accontenta del "false" di default per non rischiare che l'acqua rovini i sensori
}

```

Java

```

public class GarageMain {
    public static void main(String[] args) {

        System.out.println("--- SESSIONE QUALIFICHE (SOLE) ---");
        PreparazioneKart kartQualifica = new SetupQualificaAsciutto();
        // Il Client chiama il Template Method!
        kartQualifica.preparaPerGara();

        System.out.println("--- SESSIONE GARA (PIOGGIA) ---");
        PreparazioneKart kartGara = new SetupGaraBagnato();
        // Il Client chiama il Template Method!
        kartGara.preparaPerGara();
    }
}

```

STRATEGY

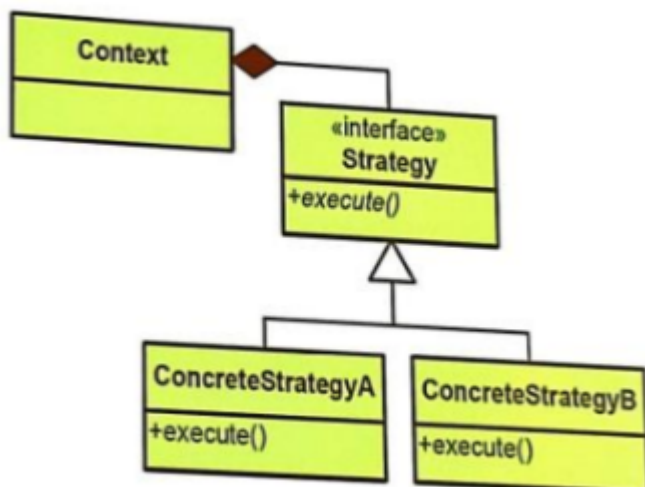
BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

L'"Algoritmo interscambiabile". Prepara diverse varianti per fare la stessa cosa e ti permette di cambiare strategia in tempo reale in base a cosa ti serve (es. su Maps cambi "Strategia" scegliendo tra a piedi, in auto o in bici e il programma cambia il calcolo al volo).

SCOPO:

Lo scopo di tale pattern è prendere una famiglia di algoritmi che fanno la stessa identica cosa, e poterli interscambiare tra di loro chiudendoli in diverse classi. L'oggetto principale che usa tali algoritmi non si accorgerà di niente se tu cambi le carte in tavola.

COME RICONOSCERLO:



Grazie al diagramma UML possiamo capire meglio come funziona. Prendendo un esempio

di paginazione di un testo, sappiamo che ci sono diversi algoritmi che lo fanno, ad esempio la paginazione a sinistra, centrale, a destra ecc.. . Se tutti questi algoritmi si facessero in un'unica classe verrebbe una classe gigantesca e rigida piena di if e switch. Se invece mettessimo questi algoritmi in classi diverse, avremo un programma migliore, ma soprattutto potremmo apportare modifiche facilmente, se ad esempio volessimo aggiungere una paginazione a zig zag, crei solo una nuova classe senza toccare quelle vecchie. Nel diagramma UML abbiamo:

-Strategy-> È l'interfaccia che definisce la regola d'oro che tutte le strategie devono avere, in questo caso è `execute()`;

-ConcreteStrategy(A e B)-> Sono le classi che contengono l'algoritmo specifico;

-**Context:** È l'oggetto principale. *Possiede* una variabile di tipo `Strategy` al suo interno (ecco il significato del rombo nero). Quando deve fare il lavoro, non fa calcoli: delega tutto a quella variabile chiamando semplicemente `suaStrategia.execute()` .

In questo pattern ci sono PRO e CONTRO:

PRO:

-Non abbiamo usato if o switch, infatti ogni algoritmo era implementato nella sua classe e poi chiamato quando e se serviva;

-Invece di creare classi rigide come `CataniaFCPossessoPalla` e `CataniaFCContropiede` , abbiamo semplicemente iniettato i comportamenti all'interno della classe `squadra`.

-Il cambio di tattica o di algoritmo avviene quando il programma è in esecuzione;

CONTRO:

-Devi conoscere gli algoritmi che devi implementare, perché se non li conosci e non sai la differenza tra di loro, il client non sceglierà per te;

-Immagina che la `Squadra` passi alla `Tattica` le statistiche di tutti e 11 i giocatori. Ma la `TatticaContropiede` magari guarda solo le statistiche dei 2 attaccanti e se ne frega dei difensori. Abbiamo sprecato memoria e calcoli per passargli dati che l'algoritmo ha ignorato, e quindi abbiamo sprecato risorse.

CODICE:

```
Java

// =====
// 1. STRATEGY (L'Interfaccia)
// =====
interface TatticaDiGioco {
    void applicaTattica();
}

// =====
// 2. CONCRETE STRATEGY A (La tattica lenta)
// =====
class TatticaPossessoPalla implements TatticaDiGioco {
    @Override
    public void applicaTattica() {
        System.out.println("⚽ I giocatori fanno girare palla lentamente, cercando spazi");
        System.out.println("🛡️ I difensori restano bassi per non subire imbucate.");
    }
}

// =====
// 3. CONCRETE STRATEGY B (La tattica veloce)
// =====
class TatticaContropiede implements TatticaDiGioco {
    @Override
    public void applicaTattica() {
        System.out.println("⚽ Recupero palla e lancio lungo immediato verso le punte!");
        System.out.println("⭐ I terzini scattano in avanti per sovrapporsi.");
    }
}
```

Qui abbiamo la strategy che è l'interfaccia che ci dice semplicemente che tutti gli algoritmi devono avere la funzione applicaTattica. Poi abbiamo i 2 algoritmi nelle classi che implementano la funzione applicaTattica.

```
// =====
// 4. CONTEXT (La Squadra in campo)
// =====
class Squadra {
    private String nome;

    // Il rombo nero: l'oggetto possiede la strategia
    private TatticaDiGioco tatticaAttuale;

    // Costruttore: la squadra scende in campo con una tattica iniziale
    public Squadra(String nome, TatticaDiGioco tatticaIniziale) {
        this.nome = nome;
        this.tatticaAttuale = tatticaIniziale;
    }

    // Il tasto sul pad per cambiare tattica in tempo reale
    public void setTattica(TatticaDiGioco nuovaTattica) {
        this.tatticaAttuale = nuovaTattica;
        System.out.println("\n⚡ [CAMBIO IN PANCHINA] Il " + this.nome + " ha cambiato s");
    }

    // L'azione in campo: deleghiamo il lavoro all'algoritmo attuale
    public void giocaPalla() {
        System.out.println("\n🏁 Inizia l'azione del " + this.nome + "...");
        tatticaAttuale.applicaTattica();
    }
}
```

Poi qui abbiamo il context che al suo interno ha una variabile di tipo strategy con cui richiama poi le varie tattiche, e poi chiama anche la funzione che c'è in strategy.

Java



```
public class PartitaMain {
    public static void main(String[] args) {

        // 1. Creiamo le nostre due tattiche (I nostri "algoritmi")
        TatticaDiGioco tikiTaka = new TatticaPossessoPalla();
        TatticaDiGioco tuttiAvanti = new TatticaContropiede();

        // 2. Il Catania scende in campo con il tiki-taka
        Squadra cataniaFC = new Squadra("Catania FC", tikiTaka);

        // Minuto 10: Il Catania prova ad attaccare
        cataniaFC.giocaPalla();

        // Minuto 80: Il Catania sta perdendo 1-0. Serve un miracolo!
        // Cambiamo la strategia a runtime!
        cataniaFC.setTattica(tuttiAvanti);

        // Minuto 85: Il Catania attacca di nuovo, ma ora il comportamento è cambiato
        cataniaFC.giocaPalla();
    }
}
```

Poi abbiamo il client, dove si setta la squadra e si mettono le tattiche.

Template Method vs Strategy

Entrambi servono a cambiare un algoritmo, ma lo fanno su due livelli completamente diversi.

- **Template Method (Livello di CLASSE):** Fissa lo "scheletro" usando l'**ereditarietà**. Ti ricordi il setup del Kart? Una volta che crei l'oggetto `SetupGaraBagnato`, è nato così e morirà così. Non puoi trasformarlo magicamente in un setup da asciutto mentre il programma gira.
- **Strategy (Livello di OGGETTO):** Usa la **composizione** (il rombo nero del diagramma). Delega il calcolo a un oggetto esterno che puoi staccare e riattaccare a piacimento. La squadra in campo può cambiare da lenta a veloce in qualsiasi secondo a *runtime*.

State vs Strategy

Questa è la domanda più insidiosa in assoluto. Se guardi i loro diagrammi UML, sono letteralmente lo stesso disegno! Un Context che delega il lavoro a un'interfaccia. Eppure, la slide dice giustamente che c'è una differenza enorme. Quale? **L'intento e chi guida il cambiamento.**

- Nello **Strategy**, è il *Client* (l'utente dal `main`) che prende la decisione e inietta la strategia da fuori. (L'allenatore decide di cambiare la tattica).
- Nello **State**, il cambiamento avviene in base alla *situazione interna* e, spesso, sono gli stessi stessi a scambiarsi il posto da soli. (Il biglietto passa automaticamente allo stato "Venduto" appena completi l'acquisto, non sei tu a forzarlo da fuori).

OBJECT NULL

BREVE DESCRIZIONE PER CAPIRLO AL VOLO:

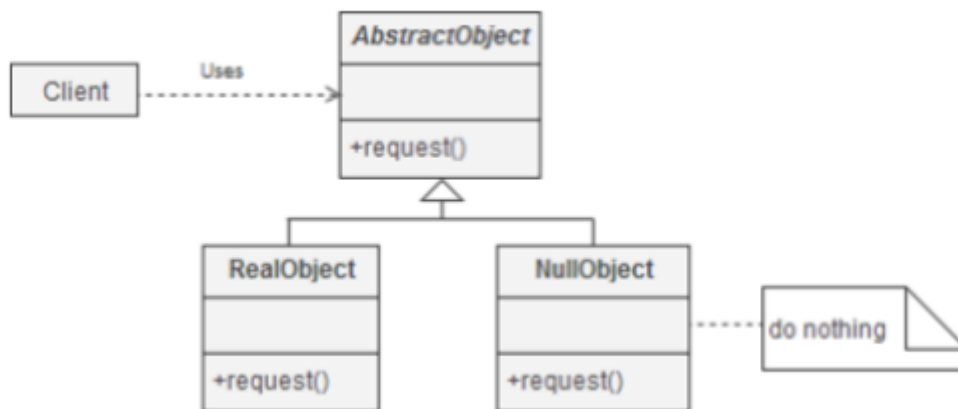
Il "Sostituto educato". Fornisce un oggetto "vuoto" che possiede comunque i metodi corretti

(che non fanno nulla), invece di restituire `null`. Evita di riempire il codice di continui controlli `if (oggetto != null)`.

SCOPO:

L'object null spesso non viene considerato neanche un pattern, ma una tecnica di refactoring per rendere il codice più pulito ed educato. Tale pattern è un sostituto di porre unavarianile a nul, quindi se un oggetto non esiste invece che mettere tanti controlli con l'if, fornisce un oggetto vuoto che ha metodi corretti. In Java, il nemico pubblico numero uno è la `NullPointerException` (l'errore che fa crashare tutto quando cerchi di usare un oggetto che in realtà è vuoto, cioè `null`). Per evitare questo crash, i programmatori riempiono il codice di controlli noiosissimi: `if (telemetria != null) { telemetria.inviaDati(); }` Per togliere tutti questi null, tale pattern crea un oggetto sostituto che ha la stessa struttura di quello vero ma non fa assolutamente nulla.

COME RICONOSCERLO:



- **AbstractObject (L'Interfaccia):** La regola base. Definisce cosa deve saper fare l'oggetto (es. il metodo `request()`).
- **RealObject (L'Oggetto Vero):** L'implementazione reale che fa il lavoro pesante (es. calcola i dati, si connette al database).
- **NullObject (Il Sostituto):** Implementa la stessa interfaccia, ma i suoi metodi sono letteralmente lasciati vuoti.

C'è però una avvertenza importante da attenzionare in tale pattern, ossia che con il null object il programma non ci darà mai errore, dato che considera l'oggetto nullo come un oggetto normale e quindi lo ignora, e magari se un'oggetto al suo interno è null e non ha niente al suo interno, noi ci metteremo ore a scoprirlo.

CODICE:

```
Java

// =====
// 1. ABSTRACT OBJECT (L'Interfaccia)
// =====
interface ModuloTelemetria {
    void inviaTempoAiBox();
}

// =====
// 2. REAL OBJECT (Il componente fisico costoso)
// =====
class TelemetriaGara implements ModuloTelemetria {
    @Override
    public void inviaTempoAiBox() {
        System.out.println("🏁 [TELEMETRIA] Connessione al server... Tempo inviato al co
    }
}

// =====
// 3. NULL OBJECT (Il "Sostituto Educatore")
// =====
class TelemetriaAssente implements ModuloTelemetria {
    @Override
    public void inviaTempoAiBox() {
        // DO NOTHING! (Non fa assolutamente nulla)
        // Niente crash, niente errori. Semplicemente ignora il comando.
    }
}
```

```
Java

class CentralinaKart {
    // La centralina ha sempre un modulo. Di default, le diamo quello vuoto!
    private ModuloTelemetria telemetria = new TelemetriaAssente();

    // Se il pilota compra l'optional, lo sostituiamo con quello vero
    public void installaTelemetria(ModuloTelemetria moduloVero) {
        this.telemetria = moduloVero;
    }

    public void tagliaTraguardo() {
        System.out.println("🏁 Giro completato!");

        // NESSUN IF! Chiamo il metodo e basta.
        // Se è il modulo vero, invierà i dati. Se è il NullObject, non succederà nulla
        telemetria.inviaTempoAiBox();
    }
}
```

Questo è il semplice codice del null object.